knn

September 8, 2026

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/datasets
--2026-09-07 11:57:28--  http://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
Resolving www.cs.toronto.edu (www.cs.toronto.edu)… 128.100.3.30
Connecting to www.cs.toronto.edu (www.cs.toronto.edu)|128.100.3.30|:80…
connected.
HTTP request sent, awaiting response… 301 Moved Permanently
Location: https://cave.cs.toronto.edu/kriz/cifar-10-python.tar.gz [following]
--2026-09-07 11:57:29--  https://cave.cs.toronto.edu/kriz/cifar-10-python.tar.gz
Resolving cave.cs.toronto.edu (cave.cs.toronto.edu)… 128.100.3.67
Connecting to cave.cs.toronto.edu (cave.cs.toronto.edu)|128.100.3.67|:443…
connected.
HTTP request sent, awaiting response… 200 OK
Length: 170498071 (163M) [application/x-gzip]
Saving to: ‘cifar-10-python.tar.gz’

cifar-10-python.tar 100%[===================>] 162.60M  91.0KB/s    in 33m 58s
```

```
2026-09-07 12:31:28 (81.7 KB/s) - ‘cifar-10-python.tar.gz’ saved
[170498071/170498071]

cifar-10-batches-py/
cifar-10-batches-py/data_batch_4
cifar-10-batches-py/readme.html
cifar-10-batches-py/test_batch
cifar-10-batches-py/data_batch_3
cifar-10-batches-py/batches.meta
cifar-10-batches-py/data_batch_2
cifar-10-batches-py/data_batch_5
cifar-10-batches-py/data_batch_1
--2026-09-07 12:31:31--  http://cs231n.stanford.edu/imagenet_val_25.npz
Resolving cs231n.stanford.edu (cs231n.stanford.edu)… 171.64.64.64
Connecting to cs231n.stanford.edu (cs231n.stanford.edu)|171.64.64.64|:80…
connected.
HTTP request sent, awaiting response… 301 Moved Permanently
Location: https://cs231n.stanford.edu/imagenet_val_25.npz [following]
--2026-09-07 12:31:32--  https://cs231n.stanford.edu/imagenet_val_25.npz
Connecting to cs231n.stanford.edu (cs231n.stanford.edu)|171.64.64.64|:443…
connected.
HTTP request sent, awaiting response… 200 OK
Length: 3940548 (3.8M)
Saving to: ‘imagenet_val_25.npz’

imagenet_val_25.npz 100%[===================>]   3.76M  2.72MB/s    in 1.4s

2026-09-07 12:31:34 (2.72 MB/s) - ‘imagenet_val_25.npz’ saved [3940548/3940548]

/content/drive/My Drive/cs231n/assignments/assignment1
```

1 k-Nearest Neighbor (kNN) exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the assignments page on the course website.

The kNN classifier consists of two stages:

- During training, the classifier takes the training data and simply remembers it
- During testing, kNN classifies every test image by comparing to all training images and transfering the labels of the k most similar training examples
- The value of k is cross-validated

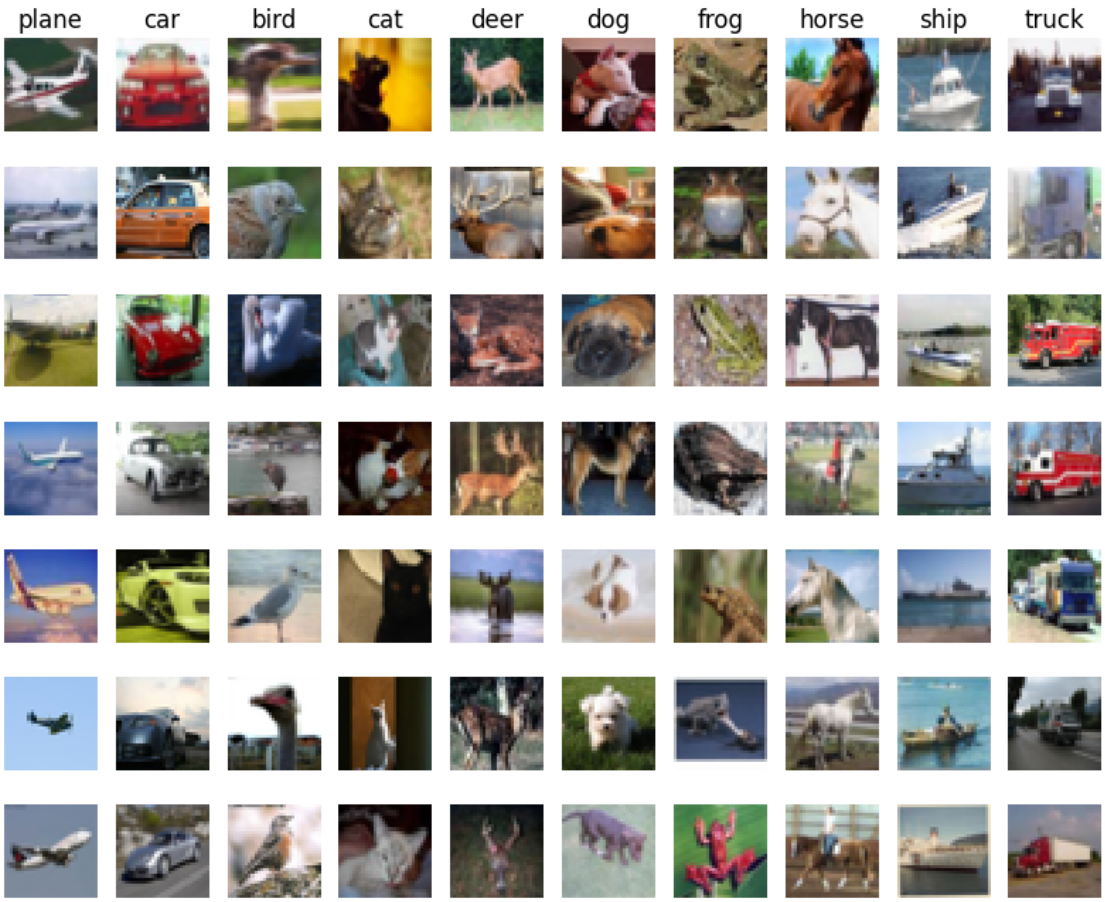
In this exercise you will implement these steps and understand the basic Image Classification pipeline, cross-validation, and gain proficiency in writing efficient, vectorized code.

[2]:
```
# Run some setup code for this notebook.

import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the␣
 ↪notebook
# rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/
 ↪autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

[3]:
```
# Load the raw CIFAR-10 data.
cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause␣
 ↪memory issue)
try:
   del X_train, y_train
   del X_test, y_test
   print('Clear previously loaded data.')
except:
   pass

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Training data shape:  (50000, 32, 32, 3)
Training labels shape:  (50000,)
Test data shape:  (10000, 32, 32, 3)
Test labels shape:  (10000,)
```

[4]:
```
# Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
 ↪'ship', 'truck']
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



[5]:
```
# Subsample the data for more efficient code execution in this exercise
num_training = 5000
mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]

num_test = 500
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]

# Reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
print(X_train.shape, X_test.shape)
```

```
(5000, 3072) (500, 3072)
```

[6]:
```
from cs231n.classifiers import KNearestNeighbor

# Create a kNN classifier instance.
# Remember that training a kNN classifier is a noop:
# the Classifier simply remembers the data and does no further processing
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
```

We would now like to classify the test data with the kNN classifier. Recall that we can break down this process into two steps:

1. First we must compute the distances between all test examples and all train examples.
2. Given these distances, for each test example we find the k nearest examples and have them vote for the label

Lets begin with computing the distance matrix between all training and test examples. For example, if there are **Ntr** training examples and **Nte** test examples, this stage should result in a **Nte x Ntr** matrix where each element (i,j) is the distance between the i-th test and j-th train example.

Note: For the three distance computations that we require you to implement in this notebook, you may not use the np.linalg.norm() function that numpy provides.

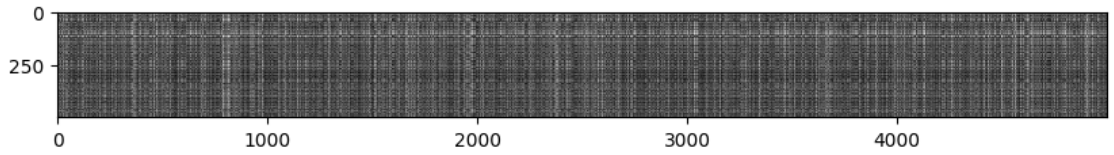
First, open `cs231n/classifiers/k_nearest_neighbor.py` and implement the function `compute_distances_two_loops` that uses a (very inefficient) double loop over all pairs of (test, train) examples and computes the distance matrix one element at a time.

[7]:
```
# Open cs231n/classifiers/k_nearest_neighbor.py and implement
# compute_distances_two_loops.

# Test your implementation:
dists = classifier.compute_distances_two_loops(X_test)
```

```
print(dists.shape)
```

```
(500, 5000)
```

[8]:
```
# We can visualize the distance matrix: each row is a single test example and
# its distances to training examples
plt.imshow(dists, interpolation='none')
plt.show()
```



Inline Question 1

Notice the structured patterns in the distance matrix, where some rows or columns are visibly brighter. (Note that with the default color scheme black indicates low distances while white indicates high distances.)

- What in the data is the cause behind the distinctly bright rows?
- What causes the columns?

Your Answer :

- **Bright rows** are caused by **outlier test images**: a row corresponds to one test point compared against *all* training points. If a test image is very different from every training image (e.g., a corrupted, over-exposed, uniformly colored, or otherwise atypical image), its L2 distance to *every* training sample is large, so the entire row appears bright.
- **Bright columns** are caused by **outlier training images**: a column corresponds to one training point compared against *all* test points. A training image that is unusual (rare appearance, heavy noise, poor lighting, etc.) lies far from every test image, so its entire column is bright. In short: bright rows/columns reveal anomalous samples in the **test set** / **training set**, respectively.

[9]:
```
# Now implement the function predict_labels and run the code below:
# We use k = 1 (which is Nearest Neighbor).
y_test_pred = classifier.predict_labels(dists, k=1)

# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 137 / 500 correct => accuracy: 0.274000
```

You should expect to see approximately 27% accuracy. Now lets try out a larger `k`, say `k = 5`:

[10]:
```
y_test_pred = classifier.predict_labels(dists, k=5)
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 139 / 500 correct => accuracy: 0.278000
```

You should expect to see a slightly better performance than with `k = 1`.

Inline Question 2

We can also use other distance metrics such as L1 distance. For pixel values $p_{ij}^{(k)}$ at location (i, j) of some image I_k,

the mean μ across all pixels over all images is

$$\mu = \frac{1}{nhw} \sum_{k=1}^{n} \sum_{i=1}^{h} \sum_{j=1}^{w} p_{ij}^{(k)}$$

And the pixel-wise mean μ_{ij} across all images is

$$\mu_{ij} = \frac{1}{n} \sum_{k=1}^{n} p_{ij}^{(k)}.$$

The general standard deviation σ and pixel-wise standard deviation σ_{ij} is defined similarly.

Which of the following preprocessing steps will not change the performance of a Nearest Neighbor classifier that uses L1 distance? Select all that apply. To clarify, both training and test examples are preprocessed in the same way.

1. Subtracting the mean μ ($\tilde{p}_{ij}^{(k)} = p_{ij}^{(k)} - \mu$.)
2. Subtracting the per pixel mean μ_{ij} ($\tilde{p}_{ij}^{(k)} = p_{ij}^{(k)} - \mu_{ij}$.)
3. Subtracting the mean μ and dividing by the standard deviation σ.
4. Subtracting the pixel-wise mean μ_{ij} and dividing by the pixel-wise standard deviation σ_{ij}.
5. Rotating the coordinate axes of the data, which means rotating all the images by the same angle. Empty regions in the image caused by rotation are padded with a same pixel value and no interpolation is performed.

YourAnswer :

Options 1, 2, and 3 do not change the performance of the L1 nearest-neighbor classifier.

YourExplanation :

Let the L1 distance between a test point x and a training point y be

$$d(x, y) = \sum_{i=1}^{D} |x_i - y_i|.$$

1. Subtracting the global mean μ — invariant

$$d(\tilde{x}, \tilde{y}) = \sum_i \left|(x_i - \mu) - (y_i - \mu)\right| = \sum_i |x_i - y_i| = d(x, y).$$

The same scalar is subtracted from both points, so it cancels pairwise.

2. Subtracting the pixel-wise mean μ_{ij} — invariant Identical argument, applied per-pixel:

$$d(\tilde{x}, \tilde{y}) = \sum_{i,j} |(p_{ij}^{(x)} - \mu_{ij}) - (p_{ij}^{(y)} - \mu_{ij})| = \sum_{i,j} |p_{ij}^{(x)} - p_{ij}^{(y)}| = d(x, y).$$

3. Subtracting μ and dividing by σ — invariant

$$d(\tilde{x}, \tilde{y}) = \sum_i \left| \frac{x_i - \mu}{\sigma} - \frac{y_i - \mu}{\sigma} \right| = \frac{1}{\sigma} \sum_i |x_i - y_i| = \frac{1}{\sigma} d(x, y).$$

Since $\sigma > 0$, **all distances are scaled by the same positive constant**. Dividing by a positive constant is a monotonic transformation, so it preserves the ordering of distances, ties, and hence each test point's nearest neighbor (arg min). The classifier's predictions — and therefore its performance — are unchanged.

4. Dividing by the pixel-wise std σ_{ij} — NOT invariant

$$d(\tilde{x}, \tilde{y}) = \sum_i \frac{|x_i - y_i|}{\sigma_i}.$$

Each dimension is re-weighted by a **different** factor $1/\sigma_i$ (dimensions with large variance get down-weighted). This changes the *relative* distances between points, so the nearest neighbor of a test point can change, and the decision boundary moves.

5. Rotating the coordinate axes — NOT invariant L1 distance is **not rotation-invariant**. For a rotation matrix R,

$$\sum_i |(R(x - y))_i| \neq \sum_i |x_i - y_i| \quad \text{in general,}$$

(e.g., the vector $(1, 1)$ has $d_{L1} = 2$, but after a 45° rotation it becomes $(\sqrt{2}, 0)$ with $d_{L1} = \sqrt{2}$). Additionally, padding empty regions with a constant value alters pixel content asymmetrically. Hence L2 would be rotation-invariant, but **L1 is not**, and performance changes.

[11]:
```
# Now lets speed up distance matrix computation by using partial vectorization
# with one loop. Implement the function compute_distances_one_loop and run the
# code below:
dists_one = classifier.compute_distances_one_loop(X_test)

# To ensure that our vectorized implementation is correct, we make sure that it
# agrees with the naive implementation. There are many ways to decide whether
# two matrices are similar; one of the simplest is the Frobenius norm. In case
# you haven't seen it before, the Frobenius norm of two matrices is the square
# root of the squared sum of differences of all elements; in other words, reshape
# the matrices into vectors and compute the Euclidean distance between them.
difference = np.linalg.norm(dists - dists_one, ord='fro')
print('One loop difference was: %f' % (difference, ))
if difference < 0.001:
```

```
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

```
One loop difference was: 0.000000
Good! The distance matrices are the same
```

[12]:
```
# Now implement the fully vectorized version inside compute_distances_no_loops
# and run the code
dists_two = classifier.compute_distances_no_loops(X_test)

# check that the distance matrix agrees with the one we computed before:
difference = np.linalg.norm(dists - dists_two, ord='fro')
print('No loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

```
No loop difference was: 0.000000
Good! The distance matrices are the same
```

[13]:
```
# Let's compare how fast the implementations are
def time_function(f, *args):
    """
    Call a function f with args and return the time (in seconds) that it took␣
 ↪to execute.
    """
    import time
    tic = time.time()
    f(*args)
    toc = time.time()
    return toc - tic

two_loop_time = time_function(classifier.compute_distances_two_loops, X_test)
print('Two loop version took %f seconds' % two_loop_time)

one_loop_time = time_function(classifier.compute_distances_one_loop, X_test)
print('One loop version took %f seconds' % one_loop_time)

no_loop_time = time_function(classifier.compute_distances_no_loops, X_test)
print('No loop version took %f seconds' % no_loop_time)

# You should see significantly faster performance with the fully vectorized␣
 ↪implementation!

# NOTE: depending on what machine you're using,
```

```
# you might not see a speedup when you go from two loops to one loop,
# and might even see a slow-down.
```

```
Two loop version took 30.415843 seconds
One loop version took 31.523510 seconds
No loop version took 0.543108 seconds
```

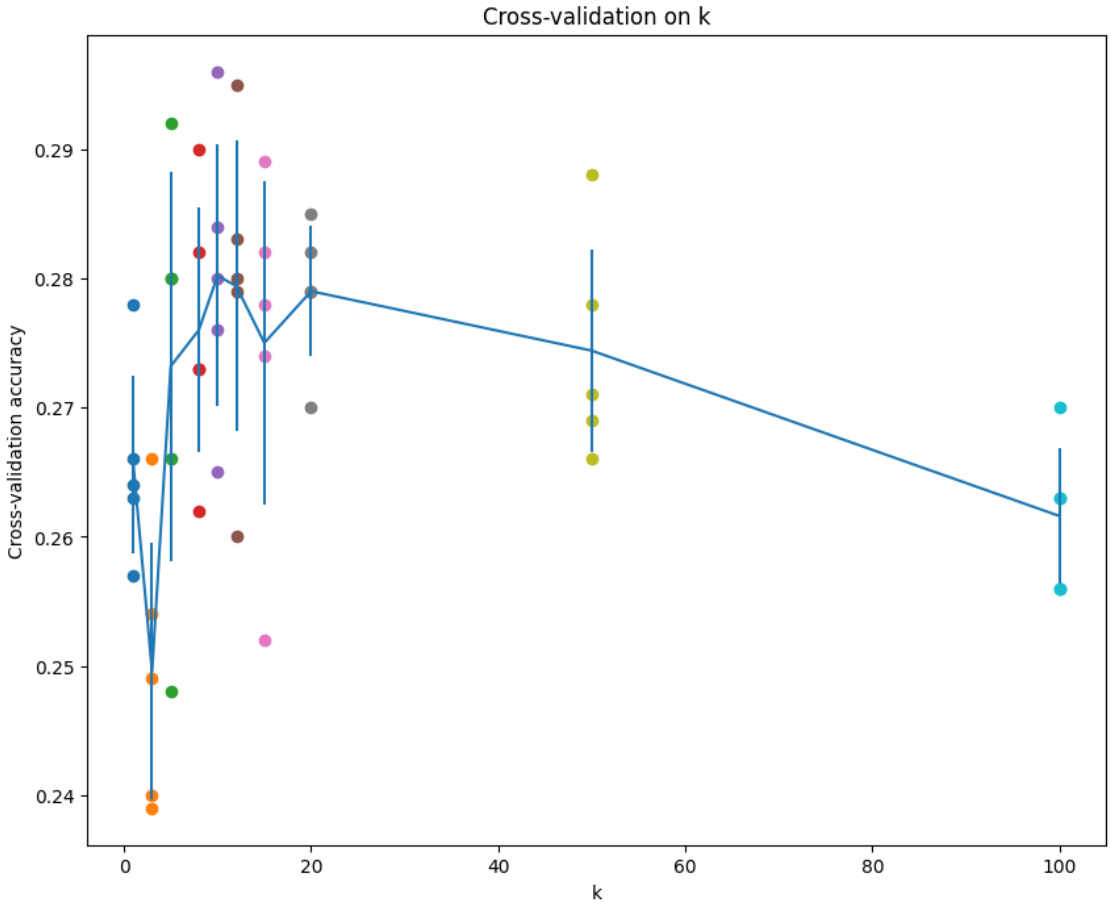
1.0.1 Cross-validation

We have implemented the k-Nearest Neighbor classifier but we set the value k = 5 arbitrarily. We will now determine the best value of this hyperparameter with cross-validation.

[16]:
```
X_train_folds = np.array_split(X_train, num_folds)
y_train_folds = np.array_split(y_train, num_folds)
k_to_accuracies = {}

for fold in range(num_folds):
    # ----    ----
    X_val  = X_train_folds[fold]
    y_val  = y_train_folds[fold]
    X_tr   = np.concatenate(X_train_folds[:fold] + X_train_folds[fold+1:],␣
 ↪axis=0)
    y_tr   = np.concatenate(y_train_folds[:fold] + y_train_folds[fold+1:],␣
 ↪axis=0)

    # ----           ----
    classifier = KNearestNeighbor()
    classifier.train(X_tr, y_tr)
    dists = classifier.compute_distances_no_loops(X_val)

    # ----    k      ----
    for k in k_choices:
        y_val_pred = classifier.predict_labels(dists, k=k)
        acc = np.mean(y_val_pred == y_val)
        k_to_accuracies.setdefault(k, []).append(acc)
```

[17]:
```
# plot the raw observations
for k in k_choices:
    accuracies = k_to_accuracies[k]
    plt.scatter([k] * len(accuracies), accuracies)

# plot the trend line with error bars that correspond to standard deviation
accuracies_mean = np.array([np.mean(v) for k,v in sorted(k_to_accuracies.
 ↪items())])
accuracies_std = np.array([np.std(v) for k,v in sorted(k_to_accuracies.
 ↪items())])
plt.errorbar(k_choices, accuracies_mean, yerr=accuracies_std)
```

```
plt.title('Cross-validation on k')
plt.xlabel('k')
plt.ylabel('Cross-validation accuracy')
plt.show()
```



[20]:
```
# Based on the cross-validation results above, choose the best value for k,
# retrain the classifier using all the training data, and test it on the test
# data. You should be able to get above 28% accuracy on the test data.
best_k = 10

classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
y_test_pred = classifier.predict(X_test, k=best_k)

# Compute and display the accuracy
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 141 / 500 correct => accuracy: 0.282000
```

Inline Question 3

Which of the following statements about k-Nearest Neighbor (k-NN) are true in a classification setting, and for all k? Select all that apply. 1. The decision boundary of the k-NN classifier is linear. 2. The training error of a 1-NN will always be lower than or equal to that of 5-NN. 3. The test error of a 1-NN will always be lower than that of a 5-NN. 4. The time needed to classify a test example with the k-NN classifier grows with the size of the training set. 5. None of the above.

YourAnswer :

Statements 2 and 4 are true; 1, 3, and 5 are false.

YourExplanation :

1. Linear decision boundary — False The k-NN decision boundary is generally highly **non-linear**. For $k = 1$ it coincides with the Voronoi diagram of the training points; for $k > 1$ it is determined by majority-vote regions. These boundaries are piecewise and can be arbitrarily complex, adapting to the local structure of the data.

2. Training error of 1-NN that of 5-NN — True Each training point's nearest neighbor is **itself** at distance 0, so 1-NN perfectly memorizes the training set and achieves (essentially) zero training error — assuming no two identical training points carry conflicting labels. For 5-NN, the vote includes 4 *other* points that may outvote the correct label, so its training error is at least that of 1-NN:

$$E_{\text{train}}(\text{1-NN}) = 0 \;\leq\; E_{\text{train}}(\text{5-NN}).$$

3. Test error of 1-NN always lower than 5-NN — False A small k makes the classifier sensitive to noise and outliers (high variance / overfitting), while a larger k smooths the decision boundary and often **generalizes better**. Which of 1-NN or 5-NN has lower test error depends entirely on the data distribution — there is no universal ordering. (In fact, on CIFAR-10 in this assignment, $k = 10$ typically beats $k = 1$.)

4. Classification time grows with training set size — True k-NN is a *lazy* learner: training is just storing the data ($O(1)$), but predicting a single test point requires computing its distance to **all** N training points:

$$\text{cost} = O(N \cdot D),$$

which grows linearly with the size of the training set. This is one of k-NN's main practical drawbacks.

5. None of the above — False Since statements 2 and 4 are true, this option is automatically false.

softmax

September 8, 2026

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/datasets
/content/drive/My Drive/cs231n/assignments/assignment1
```

1 Softmax Classifier exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the assignments page on the course website.

In this exercise you will:

- implement a fully-vectorized **loss function** for the Softmax classifier.
- implement the fully-vectorized expression for its **analytic gradient**
- **check your implementation** using numerical gradient
- use a validation set to **tune the learning rate and regularization** strength
- **optimize** the loss function with **SGD**
- **visualize** the final learned weights

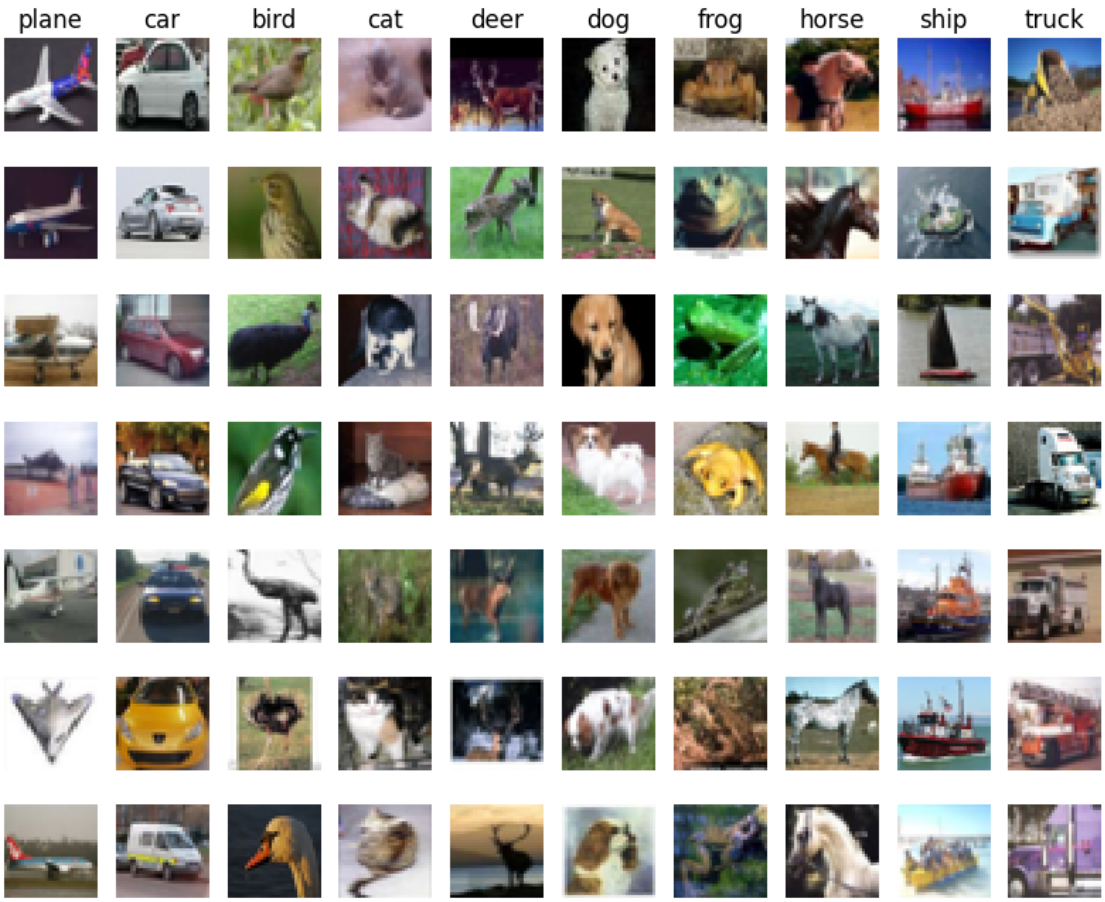
[2]:
```
# Run some setup code for this notebook.
import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
# notebook rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/
 ↪autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

1.1 CIFAR-10 Data Loading and Preprocessing

[3]:
```
# Load the raw CIFAR-10 data.
cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause␣
 ↪memory issue)
try:
   del X_train, y_train
   del X_test, y_test
   print('Clear previously loaded data.')
except:
   pass

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Training data shape:  (50000, 32, 32, 3)
Training labels shape:  (50000,)
Test data shape:  (10000, 32, 32, 3)
Test labels shape:  (10000,)
```

[4]:
```
# Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
 ↪'ship', 'truck']
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



[5]:
```
# Split the data into train, val, and test sets. In addition we will
# create a small development set as a subset of the training data;
# we can use this for development so our code runs faster.
num_training = 49000
num_validation = 1000
num_test = 1000
num_dev = 500

# Our validation set will be num_validation points from the original
# training set.
mask = range(num_training, num_training + num_validation)
X_val = X_train[mask]
y_val = y_train[mask]

# Our training set will be the first num_train points from the original
# training set.
mask = range(num_training)
X_train = X_train[mask]
y_train = y_train[mask]

# We will also make a development set, which is a small subset of
# the training set.
mask = np.random.choice(num_training, num_dev, replace=False)
X_dev = X_train[mask]
y_dev = y_train[mask]

# We use the first num_test points of the original test set as our
# test set.
mask = range(num_test)
X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

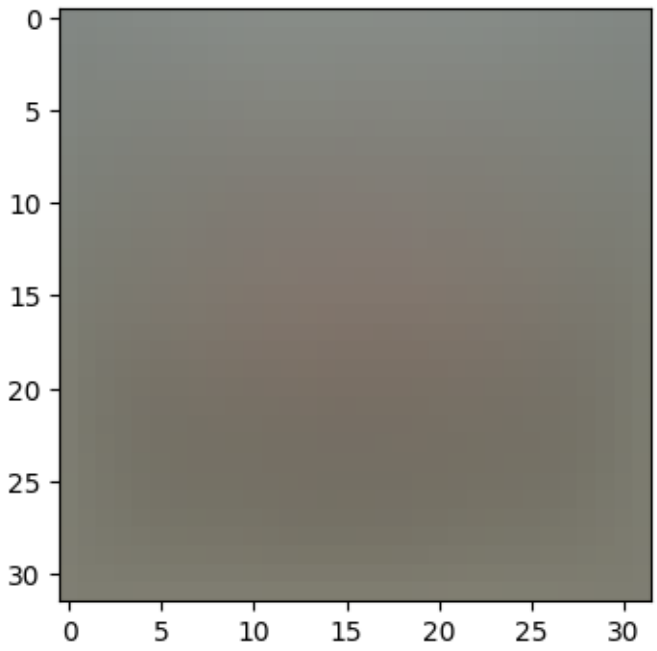
```
Train data shape:  (49000, 32, 32, 3)
Train labels shape:  (49000,)
Validation data shape:  (1000, 32, 32, 3)
Validation labels shape:  (1000,)
Test data shape:  (1000, 32, 32, 3)
Test labels shape:  (1000,)
```

[6]:
```
# Preprocessing: reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

# As a sanity check, print out the shapes of the data
print('Training data shape: ', X_train.shape)
print('Validation data shape: ', X_val.shape)
print('Test data shape: ', X_test.shape)
print('dev data shape: ', X_dev.shape)
```

```
Training data shape:  (49000, 3072)
Validation data shape:  (1000, 3072)
Test data shape:  (1000, 3072)
dev data shape:  (500, 3072)
```

[7]:
```
# Preprocessing: subtract the mean image
# first: compute the image mean based on the training data
mean_image = np.mean(X_train, axis=0)
print(mean_image[:10]) # print a few of the elements
plt.figure(figsize=(4,4))
plt.imshow(mean_image.reshape((32,32,3)).astype('uint8')) # visualize the mean␣
 ↪image
plt.show()

# second: subtract the mean image from train and test data
X_train -= mean_image
X_val -= mean_image
X_test -= mean_image
X_dev -= mean_image

# third: append the bias dimension of ones (i.e. bias trick) so that our␣
 ↪classifier
# only has to worry about optimizing a single weight matrix W.
X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

print(X_train.shape, X_val.shape, X_test.shape, X_dev.shape)
```

```
[130.64189796 135.98173469 132.47391837 130.05569388 135.34804082
 131.75402041 130.96055102 136.14328571 132.47636735 131.48467347]
```




```
(49000, 3073) (1000, 3073) (1000, 3073) (500, 3073)
```

1.2 Softmax Classifier

Your code for this section will all be written inside `cs231n/classifiers/softmax.py`.

As you can see, we have prefilled the function `softmax_loss_naive` which uses for loops to evaluate the softmax loss function.

[8]:
```
# Evaluate the naive implementation of the loss we provided for you:
from cs231n.classifiers.softmax import softmax_loss_naive
import time

# generate a random Softmax classifier weight matrix of small numbers
W = np.random.randn(3073, 10) * 0.0001

loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
print('loss: %f' % (loss, ))

# As a rough sanity check, our loss should be something close to -log(0.1).
print('loss: %f' % loss)
print('sanity check: %f' % (-np.log(0.1)))
```

```
loss: 2.317753
loss: 2.317753
sanity check: 2.302585
```

Inline Question 1

Why do we expect our loss to be close to -log(0.1)? Explain briefly.

Your Answer :

At initialization, the weights W are drawn from a Gaussian with a very small standard deviation ($0.001 \cdot \mathcal{N}(0,1)$), so the scores $s_j = x_i^\top W_j$ for all $C = 10$ classes are roughly equal and close to zero. The softmax therefore assigns approximately uniform probabilities:

$$p_j = \frac{e^{s_j}}{\sum_{k=1}^{10} e^{s_k}} \approx \frac{1}{10} = 0.1 \quad \forall j$$

The cross-entropy loss for sample i is $L_i = -\log(p_{y_i}) \approx -\log(0.1) \approx 2.303$. Intuitively, this is the loss of **random guessing** among 10 equally likely classes, so before any training the loss should start near this value.

The `grad` returned from the function above is right now all zero. Derive and implement the gradient for the softmax loss function and implement it inline inside the function `softmax_loss_naive`. You will find it helpful to interleave your new code inside the existing function.

To check that you have correctly implemented the gradient, you can numerically estimate the gradient of the loss function and compare the numeric estimate to the gradient that you computed. We have provided code that does this for you:

[9]:
```
# Once you've implemented the gradient, recompute it with the code below
# and gradient check it with the function we provided for you

# Compute the loss and its gradient at W.
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.0)

# Numerically compute the gradient along several randomly chosen dimensions, and
# compare them with your analytically computed gradient. The numbers should␣
 ↪match
# almost exactly along all dimensions.
from cs231n.gradient_check import grad_check_sparse
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad)

# do the gradient check once again with regularization turned on
# you didn't forget the regularization gradient did you?
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad)
```

```
numerical: -3.868258 analytic: -3.868259, relative error: 2.953677e-08
numerical: -0.608191 analytic: -0.608191, relative error: 3.954469e-08
numerical: -3.700340 analytic: -3.700340, relative error: 9.230148e-09
numerical: -4.922471 analytic: -4.922471, relative error: 2.135084e-09
numerical: 2.175365 analytic: 2.175365, relative error: 4.845333e-08
numerical: -1.171291 analytic: -1.171292, relative error: 3.100761e-08
```

```
numerical: -1.228377 analytic: -1.228377, relative error: 3.446901e-08
numerical: 1.042794 analytic: 1.042794, relative error: 2.180652e-08
numerical: -1.118420 analytic: -1.118420, relative error: 2.030047e-08
numerical: 0.322945 analytic: 0.322945, relative error: 2.805911e-09
numerical: 0.087009 analytic: 0.087009, relative error: 3.685556e-07
numerical: -2.798277 analytic: -2.798277, relative error: 8.147301e-09
numerical: 1.505186 analytic: 1.505186, relative error: 1.548348e-08
numerical: 0.841314 analytic: 0.841313, relative error: 5.687116e-08
numerical: 3.055925 analytic: 3.055925, relative error: 2.676781e-08
numerical: -0.163283 analytic: -0.163283, relative error: 1.263486e-08
numerical: -2.838387 analytic: -2.838387, relative error: 4.144204e-08
numerical: 2.291743 analytic: 2.291743, relative error: 2.999011e-08
numerical: -0.134256 analytic: -0.134256, relative error: 6.700084e-07
numerical: -1.717128 analytic: -1.717128, relative error: 8.451785e-08
```

Inline Question 2

Although gradcheck is reliable softmax loss, it is possible that for SVM loss, once in a while, a dimension in the gradcheck will not match exactly. What could such a discrepancy be caused by? Is it a reason for concern? What is a simple example in one dimension where a svm loss gradient check could fail? How would change the margin affect of the frequency of this happening?

Note that SVM loss for a sample (x_i, y_i) is defined as:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta)$$

where j iterates over all classes except the correct class y_i and s_j denotes the classifier score for j^{th} class. Δ is a scalar margin. For more information, refer to 'Multiclass Support Vector Machine loss' on this page.

Hint: the SVM loss function is not strictly speaking differentiable.

YourAnswer :

Cause. The SVM loss

$$L_i = \sum_{j \neq y_i} \max\left(0,\ s_j - s_{y_i} + \Delta\right)$$

is **piecewise linear** and is **not differentiable at the "kinks"** where $s_j - s_{y_i} + \Delta = 0$. The numerical gradient uses a central difference, which averages the left and right derivatives:

$$\frac{L(s+h) - L(s-h)}{2h} = \frac{1}{2}\left(\frac{\partial L^-}{\partial s} + \frac{\partial L^+}{\partial s}\right),$$

whereas the analytic gradient picks a single subgradient via a one-sided convention (e.g., $\mathbb{1}[s_j - s_{y_i} + \Delta > 0]$). When the evaluation point sits (nearly) exactly on a kink, these two values disagree, and the check reports a mismatch in that dimension. **Is it a concern?** No. It is a well-understood artifact of non-differentiability, not a bug — with random data the probability of landing exactly on a kink is negligible, so an occasional tiny mismatch can be safely ignored. **1-D example.** Let the loss depend on a single scalar score s (the wrong-class score), with $s_y = 0$ fixed and $\Delta = 1$:

$$L(s) = \max(0,\ s + 1).$$

The true derivative is 0 for $s < -1$, 1 for $s > -1$, and **undefined at** $s = -1$. Evaluating at $s = -1$: - Analytic gradient (convention $\mathbb{1}[s + 1 > 0]$): margin $= 0$, so gradient $= 0$. - Numerical gradient:

$$\frac{L(-1+h) - L(-1-h)}{2h} = \frac{h - 0}{2h} = 0.5.$$

The check fails with a discrepancy of exactly 0.5 (the average of the two one-sided derivatives). **Effect of the margin.** The kink lies where $s_j - s_{y_i} = -\Delta$. With randomly initialized weights the scores cluster near zero, so **increasing Δ pushes the kinks further away from typical score values**, making it *less* likely that a sampled point lands within the numerical step size h of a kink — the frequency of mismatches decreases. Conversely, a **smaller margin moves kinks toward** $s_j \approx s_{y_i}$, which is where random scores are concentrated, *increasing* the frequency of gradcheck failures.

[10]:
```
# Next implement the function softmax_loss_vectorized; for now only compute the loss;
# we will implement the gradient in a moment.
tic = time.time()
loss_naive, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss: %e computed in %fs' % (loss_naive, toc - tic))

from cs231n.classifiers.softmax import softmax_loss_vectorized
tic = time.time()
loss_vectorized, _ = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic))

# The losses should match but your vectorized implementation should be much faster.
print('difference: %f' % (loss_naive - loss_vectorized))
```

```
Naive loss: 2.317753e+00 computed in 0.069864s
Vectorized loss: 2.317753e+00 computed in 0.019366s
difference: 0.000000
```

[11]:
```
# Complete the implementation of softmax_loss_vectorized, and compute the gradient
# of the loss function in a vectorized way.

# The naive implementation and the vectorized implementation should match, but
# the vectorized version should still be much faster.
tic = time.time()
_, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss and gradient: computed in %fs' % (toc - tic))

tic = time.time()
_, grad_vectorized = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
```

```
toc = time.time()
print('Vectorized loss and gradient: computed in %fs' % (toc - tic))

# The loss is a single number, so it is easy to compare the values computed
# by the two implementations. The gradient on the other hand is a matrix, so
# we use the Frobenius norm to compare them.
difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('difference: %f' % difference)
```

```
Naive loss and gradient: computed in 0.039108s
Vectorized loss and gradient: computed in 0.008106s
difference: 0.000000
```

1.2.1 Stochastic Gradient Descent

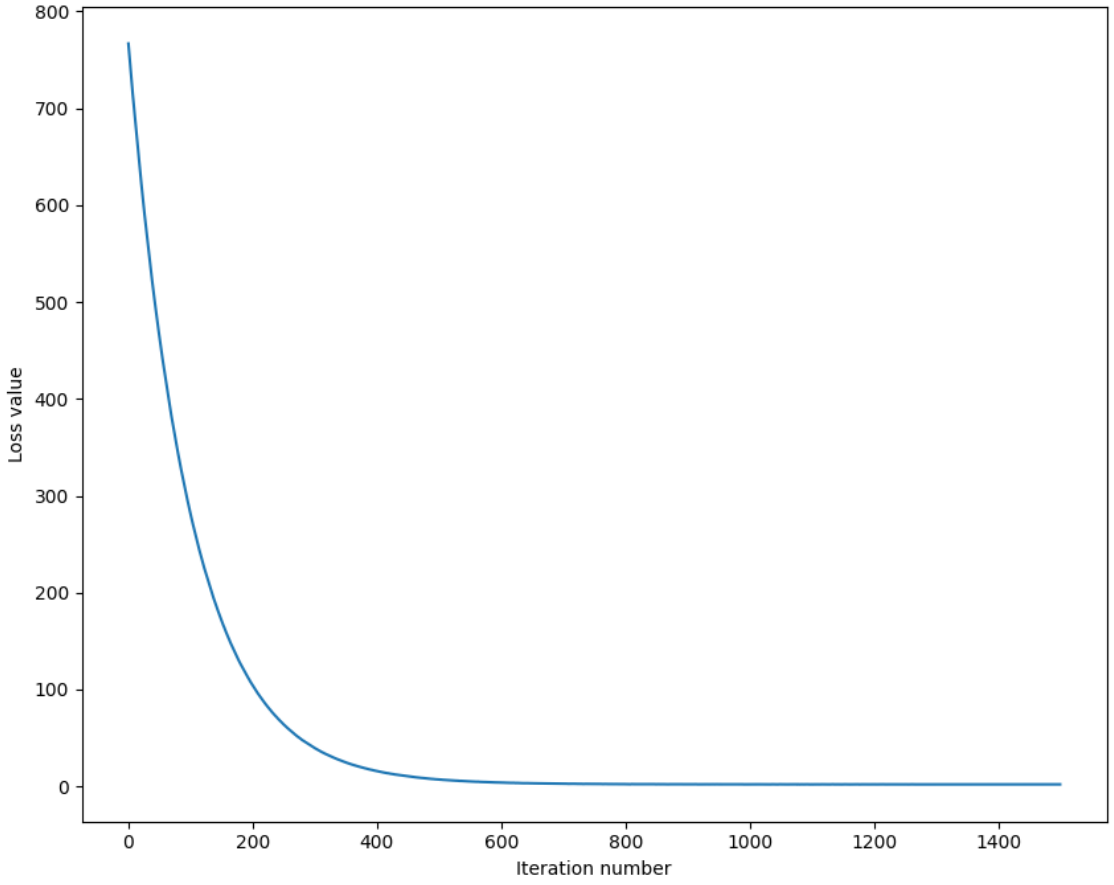
We now have vectorized and efficient expressions for the loss, the gradient and our gradient matches the numerical gradient. We are therefore ready to do SGD to minimize the loss. Your code for this part will be written inside `cs231n/classifiers/linear_classifier.py`.

[12]:
```
# In the file linear_classifier.py, implement SGD in the function
# LinearClassifier.train() and then run it with the code below.
from cs231n.classifiers import Softmax
softmax = Softmax()
tic = time.time()
loss_hist = softmax.train(X_train, y_train, learning_rate=1e-7, reg=2.5e4,
                      num_iters=1500, verbose=True)
toc = time.time()
print('That took %fs' % (toc - tic))
```

```
iteration 0 / 1500: loss 766.882620
iteration 100 / 1500: loss 281.252926
iteration 200 / 1500: loss 104.136409
iteration 300 / 1500: loss 39.547251
iteration 400 / 1500: loss 15.767236
iteration 500 / 1500: loss 7.097251
iteration 600 / 1500: loss 3.893136
iteration 700 / 1500: loss 2.810045
iteration 800 / 1500: loss 2.343305
iteration 900 / 1500: loss 2.145460
iteration 1000 / 1500: loss 2.070942
iteration 1100 / 1500: loss 2.093807
iteration 1200 / 1500: loss 2.116384
iteration 1300 / 1500: loss 2.036166
iteration 1400 / 1500: loss 2.088424
That took 6.248016s
```

[13]:
```
# A useful debugging strategy is to plot the loss as a function of
# iteration number:
```

```
plt.plot(loss_hist)
plt.xlabel('Iteration number')
plt.ylabel('Loss value')
plt.show()
```



[14]:
```
# Write the LinearClassifier.predict function and evaluate the performance on
# both the training and validation set
# You should get validation accuracy of about 0.34 (> 0.33).
y_train_pred = softmax.predict(X_train)
print('training accuracy: %f' % (np.mean(y_train == y_train_pred), ))
y_val_pred = softmax.predict(X_val)
print('validation accuracy: %f' % (np.mean(y_val == y_val_pred), ))
```

```
training accuracy: 0.326653
validation accuracy: 0.338000
```

[15]:
```
# Save the trained model for autograder.
softmax.save("softmax.npy")
```

```
softmax.npy saved.
```

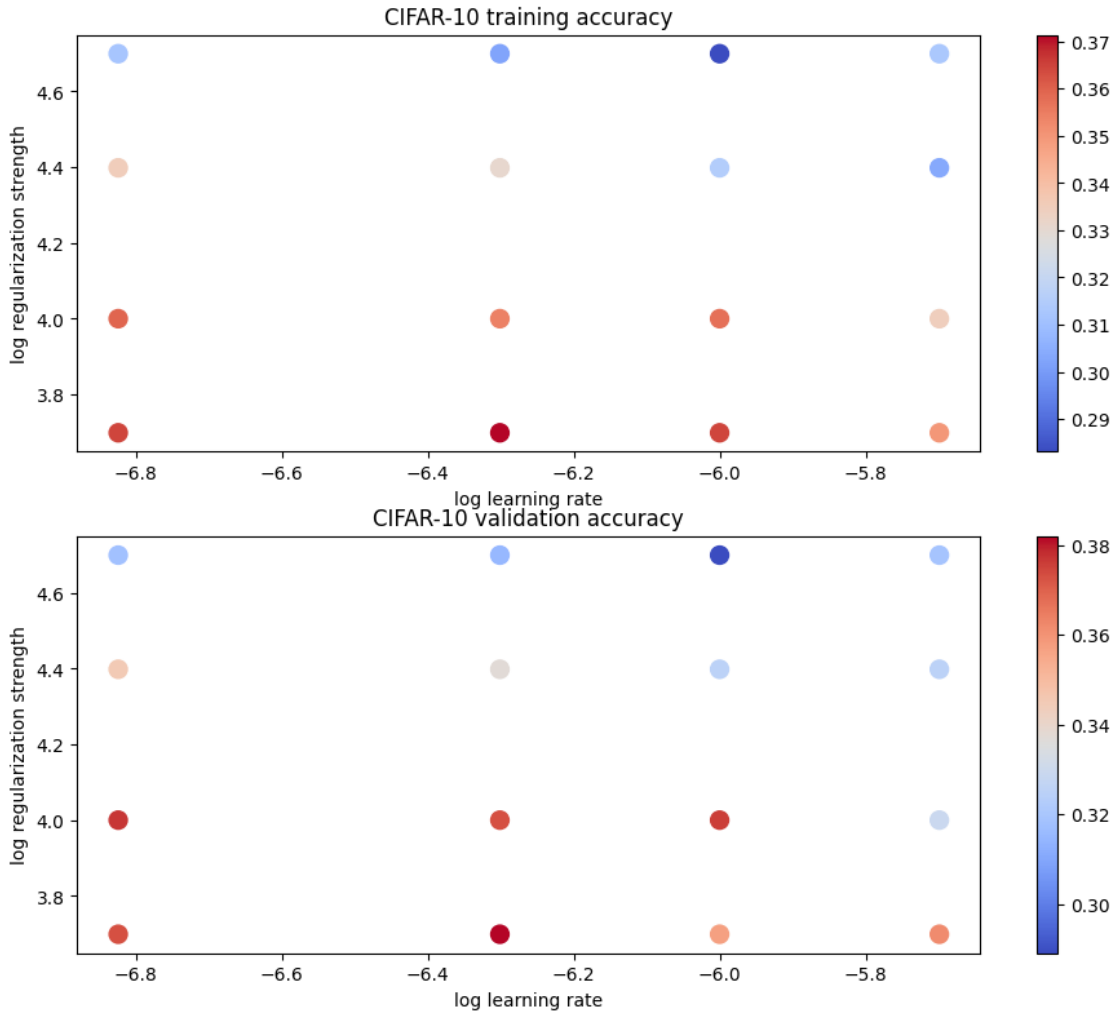
[16]:
```
# Use the validation set to tune hyperparameters (regularization strength and
# learning rate). You should experiment with different ranges for the learning
# rates and regularization strengths; if you are careful you should be able to
# get a classification accuracy of about 0.365 (> 0.36) on the validation set.

# results is dictionary mapping tuples of the form
# (learning_rate, regularization_strength) to tuples of the form
# (training_accuracy, validation_accuracy). The accuracy is simply the fraction
# of data points that are correctly classified.
results = {}
best_val = -1        # The highest validation accuracy that we have seen so far.
best_softmax = None  # The Softmax object that achieved the highest validation␣
 ↪rate.

################################################################################
# TODO:                                                                        #
# Write code that chooses the best hyperparameters by tuning on the validation #
# set. For each combination of hyperparameters, train a Softmax on the.        #
# training set, compute its accuracy on the training and validation sets, and  #
# store these numbers in the results dictionary. In addition, store the best   #
# validation accuracy in best_val and the Softmax object that achieves this.   #
################################################################################

#            0.365
learning_rates = [1.5e-7, 5e-7, 1e-6, 2e-6]
regularization_strengths = [5e3, 1e4, 2.5e4, 5e4]

#      num_iters=500          1500
num_iters = 1500

for lr in learning_rates:
    for reg in regularization_strengths:
        # 1.     Softmax
        softmax = Softmax()
        softmax.train(X_train, y_train,
                      learning_rate=lr,
                      reg=reg,
                      num_iters=num_iters,
                      verbose=False)

        # 2.
        y_train_pred = softmax.predict(X_train)
        y_val_pred   = softmax.predict(X_val)

        # 3.
        train_accuracy = np.mean(y_train_pred == y_train)
        val_accuracy   = np.mean(y_val_pred == y_val)
```

```
        # 4.   results
        results[(lr, reg)] = (train_accuracy, val_accuracy)

        # 5.
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_softmax = softmax

################################################################################
#                               END OF YOUR CODE                               #
################################################################################

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
                lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved during cross-validation: %f' %␣
 ↪best_val)
```

```
lr 1.500000e-07 reg 5.000000e+03 train accuracy: 0.364531 val accuracy: 0.373000
lr 1.500000e-07 reg 1.000000e+04 train accuracy: 0.359571 val accuracy: 0.377000
lr 1.500000e-07 reg 2.500000e+04 train accuracy: 0.334939 val accuracy: 0.345000
lr 1.500000e-07 reg 5.000000e+04 train accuracy: 0.311980 val accuracy: 0.318000
lr 5.000000e-07 reg 5.000000e+03 train accuracy: 0.371367 val accuracy: 0.382000
lr 5.000000e-07 reg 1.000000e+04 train accuracy: 0.354429 val accuracy: 0.373000
lr 5.000000e-07 reg 2.500000e+04 train accuracy: 0.330918 val accuracy: 0.337000
lr 5.000000e-07 reg 5.000000e+04 train accuracy: 0.302163 val accuracy: 0.315000
lr 1.000000e-06 reg 5.000000e+03 train accuracy: 0.364551 val accuracy: 0.357000
lr 1.000000e-06 reg 1.000000e+04 train accuracy: 0.357347 val accuracy: 0.376000
lr 1.000000e-06 reg 2.500000e+04 train accuracy: 0.315061 val accuracy: 0.325000
lr 1.000000e-06 reg 5.000000e+04 train accuracy: 0.283082 val accuracy: 0.289000
lr 2.000000e-06 reg 5.000000e+03 train accuracy: 0.349694 val accuracy: 0.362000
lr 2.000000e-06 reg 1.000000e+04 train accuracy: 0.334551 val accuracy: 0.329000
lr 2.000000e-06 reg 2.500000e+04 train accuracy: 0.303612 val accuracy: 0.325000
lr 2.000000e-06 reg 5.000000e+04 train accuracy: 0.312878 val accuracy: 0.319000
best validation accuracy achieved during cross-validation: 0.382000
```

[17]:
```
# Visualize the cross-validation results
import math
import pdb

# pdb.set_trace()

x_scatter = [math.log10(x[0]) for x in results]
```

```
y_scatter = [math.log10(x[1]) for x in results]

# plot training accuracy
marker_size = 100
colors = [results[x][0] for x in results]
plt.subplot(2, 1, 1)
plt.tight_layout(pad=3)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 training accuracy')

# plot validation accuracy
colors = [results[x][1] for x in results] # default size of markers is 20
plt.subplot(2, 1, 2)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 validation accuracy')
plt.show()
```



[18]:
```
# Evaluate the best softmax on test set
y_test_pred = best_softmax.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print('Softmax classifier on raw pixels final test set accuracy: %f' %␣
  ↪test_accuracy)
```

```
Softmax classifier on raw pixels final test set accuracy: 0.382000
```

[19]:
```
# Save best softmax model
best_softmax.save("best_softmax.npy")
```

```
best_softmax.npy saved.
```

[20]:
```
# Visualize the learned weights for each class.
# Depending on your choice of learning rate and regularization strength, these␣
  ↪may
```

```
# or may not be nice to look at.
w = best_softmax.W[:-1,:] # strip out the bias
w = w.reshape(32, 32, 3, 10)
w_min, w_max = np.min(w), np.max(w)
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
 ↪'ship', 'truck']
for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Rescale the weights to be between 0 and 255
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype('uint8'))
    plt.axis('off')
    plt.title(classes[i])
```

Inline question 3

Describe what your visualized Softmax classifier weights look like, and offer a brief explanation for why they look the way they do.

YourAnswer :

Each column of W (visualized per class) looks like a **noisy, blurry template** that vaguely resembles the average appearance of that class — e.g., the *horse* template shows a reddish/purple silhouette on a blue–green background, and the *car/ship* templates show gray, centered blob-like

shapes. This happens because a linear classifier computes a single dot product per class,

$$s_c = \sum_d W_{d,c}\, x_d,$$

which is maximized when the image pixels align with the weight template $W_{:,c}$. During training, images of the same class share strong low-level statistics (dominant background colors such as sky-blue or grass-green, plus characteristic silhouettes), so gradient descent pushes $W_{:,c}$ toward the **mean appearance of class** c. The templates are noisy/blurry rather than sharp because (i) a single linear template can only capture coarse, averaged structure, and (ii) CIFAR-10 images of one class vary enormously in pose, scale, and viewpoint, so the model can only learn shared color/layout patterns, not detailed object features.

Inline Question 4 - *True or False*

Suppose the overall training loss is defined as the sum of the per-datapoint loss over all training examples. It is possible to add a new datapoint to a training set that would change the softmax loss, but leave the SVM loss unchanged.

YourAnswer :

True.

YourExplanation :

The per-datapoint SVM loss is bounded below by 0 and **can be exactly zero** whenever the correct score beats every wrong score by at least the margin:

$$\sum_{j \neq y_i} \max\left(0,\ s_j - s_{y_i} + \Delta\right) = 0 \quad \text{if} \quad s_{y_i} \geq s_j + \Delta \ \ \forall j \neq y_i.$$

In contrast, the softmax cross-entropy loss is **strictly positive** for any finite scores:

$$L_i^{\text{softmax}} = -\log \frac{e^{s_{y_i}}}{\sum_j e^{s_j}} = -\log p_{y_i} > 0 \quad \text{since } p_{y_i} < 1 \text{ always.}$$

Concrete example. Let $\Delta = 1$ and suppose a new datapoint has scores $s_y = 10$ and $s_j = -10$ for all $j \neq y$. Then: - **SVM:** every hinge term is $\max(0, -10 - 10 + 1) = \max(0, -19) = 0$, so the datapoint adds **exactly** 0 to the total SVM loss. - **Softmax:**

$$L_i = -\log \frac{e^{10}}{e^{10} + 9e^{-10}} = \log(1 + 9e^{-20}) \approx 9e^{-20} > 0,$$

so the total softmax loss **strictly increases** by this (tiny but nonzero) amount. Therefore a datapoint classified with a sufficiently large margin leaves the SVM loss unchanged while always increasing the softmax loss.

[ ]:

two_layer_net

September 8, 2026

[ ]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/datasets
/content/drive/My Drive/cs231n/assignments/assignment1
```

1 Fully-Connected Neural Nets

In this exercise we will implement fully-connected networks using a modular approach. For each layer we will implement a `forward` and a `backward` function. The `forward` function will receive inputs, weights, and other parameters and will return both an output and a `cache` object storing data needed for the backward pass, like this:

```
def layer_forward(x, w):
  """ Receive inputs x and weights w """
  # Do some computations ...
  z = # ... some intermediate value
  # Do some more computations ...
  out = # the output
```

```
  cache = (x, w, z, out) # Values we need to compute gradients

  return out, cache
```

The backward pass will receive upstream derivatives and the `cache` object, and will return gradients with respect to the inputs and weights, like this:

```
def layer_backward(dout, cache):
  """
  Receive dout (derivative of loss with respect to outputs) and cache,
  and compute derivative with respect to inputs.
  """
  # Unpack cache values
  x, w, z, out = cache

  # Use values in cache to compute derivatives
  dx = # Derivative of loss with respect to x
  dw = # Derivative of loss with respect to w

  return dx, dw
```

After implementing a bunch of layers this way, we will be able to easily combine them to build classifiers with different architectures.

[ ]:
```
# As usual, a bit of setup
from __future__ import print_function
import time
import numpy as np
import matplotlib.pyplot as plt
from cs231n.classifiers.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient,␣
 ↪eval_numerical_gradient_array
from cs231n.solver import Solver

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/
 ↪autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2

def rel_error(x, y):
  """ returns relative error """
```

```
  return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

[ ]:
```
# Load the (preprocessed) CIFAR10 data.

data = get_CIFAR10_data()
for k, v in list(data.items()):
  print(('%s: ' % k, v.shape))
```

```
('X_train: ', (49000, 3, 32, 32))
('y_train: ', (49000,))
('X_val: ', (1000, 3, 32, 32))
('y_val: ', (1000,))
('X_test: ', (1000, 3, 32, 32))
('y_test: ', (1000,))
```

2 Affine layer: forward

Open the file `cs231n/layers.py` and implement the `affine_forward` function.

Once you are done you can test your implementaion by running the following:

[ ]:
```
# Test the affine_forward function

num_inputs = 2
input_shape = (4, 5, 6)
output_dim = 3

input_size = num_inputs * np.prod(input_shape)
weight_size = output_dim * np.prod(input_shape)

x = np.linspace(-0.1, 0.5, num=input_size).reshape(num_inputs, *input_shape)
w = np.linspace(-0.2, 0.3, num=weight_size).reshape(np.prod(input_shape),␣
 ↪output_dim)
b = np.linspace(-0.3, 0.1, num=output_dim)

out, _ = affine_forward(x, w, b)
correct_out = np.array([[ 1.49834967,  1.70660132,  1.91485297],
                        [ 3.25553199,  3.5141327,   3.77273342]])

# Compare your output with ours. The error should be around e-9 or less.
print('Testing affine_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing affine_forward function:
difference:  9.769849468192957e-10
```

3 Affine layer: backward

Now implement the `affine_backward` function and test your implementation using numeric gradient checking.

[ ]:
```
# Test the affine_backward function
np.random.seed(231)
x = np.random.randn(10, 2, 3)
w = np.random.randn(6, 5)
b = np.random.randn(5)
dout = np.random.randn(10, 5)

dx_num = eval_numerical_gradient_array(lambda x: affine_forward(x, w, b)[0], x,␣
 ↪dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_forward(x, w, b)[0], w,␣
 ↪dout)
db_num = eval_numerical_gradient_array(lambda b: affine_forward(x, w, b)[0], b,␣
 ↪dout)

_, cache = affine_forward(x, w, b)
dx, dw, db = affine_backward(dout, cache)

# The error should be around e-10 or less
print('Testing affine_backward function:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_backward function:
dx error:  5.399100368651805e-11
dw error:  9.904211865398145e-11
db error:  2.4122867568119087e-11
```

4 ReLU activation: forward

Implement the forward pass for the ReLU activation function in the `relu_forward` function and test your implementation using the following:

[ ]:
```
# Test the relu_forward function

x = np.linspace(-0.5, 0.5, num=12).reshape(3, 4)

out, _ = relu_forward(x)
correct_out = np.array([[ 0.,          0.,          0.,          0.,        ],
                        [ 0.,          0.,          0.04545455,  0.13636364,],
                        [ 0.22727273,  0.31818182,  0.40909091,  0.5,       ]])
```

```
# Compare your output with ours. The error should be on the order of e-8
print('Testing relu_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing relu_forward function:
difference:  4.999999798022158e-08
```

5 ReLU activation: backward

Now implement the backward pass for the ReLU activation function in the `relu_backward` function and test your implementation using numeric gradient checking:

[ ]:
```
np.random.seed(231)
x = np.random.randn(10, 10)
dout = np.random.randn(*x.shape)

dx_num = eval_numerical_gradient_array(lambda x: relu_forward(x)[0], x, dout)

_, cache = relu_forward(x)
dx = relu_backward(dout, cache)

# The error should be on the order of e-12
print('Testing relu_backward function:')
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing relu_backward function:
dx error:  3.2756349136310288e-12
```

5.1 Inline Question 1:

We've only asked you to implement ReLU, but there are a number of different activation functions that one could use in neural networks, each with its pros and cons. In particular, an issue commonly seen with activation functions is getting zero (or close to zero) gradient flow during backpropagation. Which of the following activation functions have this problem? If you consider these functions in the one dimensional case, what types of input would lead to this behaviour? 1. Sigmoid 2. ReLU 3. Leaky ReLU

Your Answer :

Both **Sigmoid** and **ReLU** suffer from the zero (or near-zero) gradient problem; **Leaky ReLU does not**.

Activation	Vanishing gradient?	Problematic inputs (1-D case)
Sigmoid	Yes (near-zero)	Large-magnitude inputs, i.e. $x \gg 0$ or $x \ll 0$
ReLU	Yes (exactly zero)	All negative inputs, i.e. $x < 0$
Leaky ReLU	No	None — gradient never vanishes

Sigmoid. With $\sigma(x) = \frac{1}{1+e^{-x}}$, the derivative is

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)).$$

This is at most 0.25 (at $x = 0$) and decays exponentially toward 0 as $x \to +\infty$ or $x \to -\infty$. So when a neuron's input lies in the **saturated regime** (very large positive or very large negative pre-activation), the local gradient is close to zero and backpropagated signals effectively die. Moreover, the maximum derivative of 0.25 means gradients shrink multiplicatively across layers even in the best case. **ReLU.** With $\mathrm{ReLU}(x) = \max(0, x)$, the derivative is

$$\mathrm{ReLU}'(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$$

so **every negative input produces exactly zero gradient**. If a unit's weights are such that its pre-activation is negative for all training examples, no gradient ever flows to that unit — the well-known **"dead ReLU"** problem, and the unit can never recover through gradient updates. **Leaky ReLU.** With $\mathrm{LeakyReLU}(x) = \max(\alpha x, x)$ (e.g. $\alpha = 0.1$), the derivative is

$$\mathrm{LeakyReLU}'(x) = \begin{cases} 1, & x > 0 \\ \alpha, & x < 0 \end{cases}$$

which is **never zero** for any input. A small but nonzero slope α keeps gradient flowing even through inactive units, so this activation avoids the vanishing-gradient problem by design.

6 "Sandwich" layers

There are some common patterns of layers that are frequently used in neural nets. For example, affine layers are frequently followed by a ReLU nonlinearity. To make these common patterns easy, we define several convenience layers in the file `cs231n/layer_utils.py`.

For now take a look at the `affine_relu_forward` and `affine_relu_backward` functions, and run the following to numerically gradient check the backward pass:

[ ]:
```
from cs231n.layer_utils import affine_relu_forward, affine_relu_backward
np.random.seed(231)
x = np.random.randn(2, 3, 4)
w = np.random.randn(12, 10)
b = np.random.randn(10)
dout = np.random.randn(2, 10)

out, cache = affine_relu_forward(x, w, b)
dx, dw, db = affine_relu_backward(dout, cache)

dx_num = eval_numerical_gradient_array(lambda x: affine_relu_forward(x, w,␣
 ↪b)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_relu_forward(x, w,␣
 ↪b)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b: affine_relu_forward(x, w,␣
 ↪b)[0], b, dout)
```

```
# Relative error should be around e-10 or less
print('Testing affine_relu_forward and affine_relu_backward:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_relu_forward and affine_relu_backward:
dx error:  2.299579177309368e-11
dw error:  8.162011105764925e-11
db error:  7.826724021458994e-12
```

7 Loss layers: Softmax

Now implement the loss and gradient for softmax in the `softmax_loss` function in `cs231n/layers.py`. These should be similar to what you implemented in `cs231n/classifiers/softmax.py`. Other loss functions (e.g. `svm_loss`) can also be implemented in a modular way, however, it is not required for this assignment.

You can make sure that the implementations are correct by running the following:

[ ]:
```
np.random.seed(231)
num_classes, num_inputs = 10, 50
x = 0.001 * np.random.randn(num_inputs, num_classes)
y = np.random.randint(num_classes, size=num_inputs)


dx_num = eval_numerical_gradient(lambda x: softmax_loss(x, y)[0], x,␣
 ↪verbose=False)
loss, dx = softmax_loss(x, y)

# Test softmax_loss function. Loss should be close to 2.3 and dx error should␣
 ↪be around e-8
print('\nTesting softmax_loss:')
print('loss: ', loss)
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing softmax_loss:
loss:  2.302545844500738
dx error:  9.384673161989355e-09
```

8 Two-layer network

Open the file `cs231n/classifiers/fc_net.py` and complete the implementation of the `TwoLayerNet` class. Read through it to make sure you understand the API. You can run the cell below to test your implementation.

[ ]:
```
np.random.seed(231)
N, D, H, C = 3, 5, 50, 7
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

std = 1e-3
model = TwoLayerNet(input_dim=D, hidden_dim=H, num_classes=C, weight_scale=std)

print('Testing initialization ... ')
W1_std = abs(model.params['W1'].std() - std)
b1 = model.params['b1']
W2_std = abs(model.params['W2'].std() - std)
b2 = model.params['b2']
assert W1_std < std / 10, 'First layer weights do not seem right'
assert np.all(b1 == 0), 'First layer biases do not seem right'
assert W2_std < std / 10, 'Second layer weights do not seem right'
assert np.all(b2 == 0), 'Second layer biases do not seem right'

print('Testing test-time forward pass ... ')
model.params['W1'] = np.linspace(-0.7, 0.3, num=D*H).reshape(D, H)
model.params['b1'] = np.linspace(-0.1, 0.9, num=H)
model.params['W2'] = np.linspace(-0.3, 0.4, num=H*C).reshape(H, C)
model.params['b2'] = np.linspace(-0.9, 0.1, num=C)
X = np.linspace(-5.5, 4.5, num=N*D).reshape(D, N).T
scores = model.loss(X)
correct_scores = np.asarray(
  [[11.53165108,  12.2917344,   13.05181771,  13.81190102,  14.57198434, 15.
 ↪33206765,  16.09215096],
   [12.05769098,  12.74614105,  13.43459113,  14.1230412,   14.81149128, 15.
 ↪49994135,  16.18839143],
   [12.58373087,  13.20054771,  13.81736455,  14.43418138,  15.05099822, 15.
 ↪66781506,  16.2846319 ]])
scores_diff = np.abs(scores - correct_scores).sum()
assert scores_diff < 1e-6, 'Problem with test-time forward pass'

print('Testing training loss (no regularization)')
y = np.asarray([0, 5, 1])
loss, grads = model.loss(X, y)
correct_loss = 3.4702243556
assert abs(loss - correct_loss) < 1e-10, 'Problem with training-time loss'

model.reg = 1.0
loss, grads = model.loss(X, y)
correct_loss = 26.5948426952
assert abs(loss - correct_loss) < 1e-10, 'Problem with regularization loss'

# Errors should be around e-7 or less
```

```
for reg in [0.0, 0.7]:
  print('Running numeric gradient check with reg = ', reg)
  model.reg = reg
  loss, grads = model.loss(X, y)

  for name in sorted(grads):
    f = lambda _: model.loss(X, y)[0]
    grad_num = eval_numerical_gradient(f, model.params[name], verbose=False)
    print('%s relative error: %.2e' % (name, rel_error(grad_num, grads[name])))
```

```
Testing initialization …
Testing test-time forward pass …
Testing training loss (no regularization)
Running numeric gradient check with reg =  0.0
W1 relative error: 1.83e-08
W2 relative error: 3.12e-10
b1 relative error: 9.83e-09
b2 relative error: 4.33e-10
Running numeric gradient check with reg =  0.7
W1 relative error: 2.53e-07
W2 relative error: 2.85e-08
b1 relative error: 1.56e-08
b2 relative error: 7.76e-10
```

9 Solver

Open the file `cs231n/solver.py` and read through it to familiarize yourself with the API. After doing so, use a `Solver` instance to train a `TwoLayerNet` that achieves about `36%` accuracy on the validation set.

[ ]:
```
input_size = 32 * 32 * 3
hidden_size = 50
num_classes = 10
model = TwoLayerNet(input_size, hidden_size, num_classes)
solver = None

################################################################################
# TODO: Use a Solver instance to train a TwoLayerNet that achieves about 36% #
# accuracy on the validation set.                                              #
################################################################################
solver = Solver(
    model, data,
    update_rule='sgd',
    optim_config={
        'learning_rate': 1e-3,
    },
    lr_decay=0.95,
```

```
    num_epochs=10,
    batch_size=200,
    print_every=100,
    verbose=True,
)
solver.train()
################################################################################
#                                END OF YOUR CODE                              #
################################################################################
```

```
(Iteration 1 / 2450) loss: 2.301725
(Epoch 0 / 10) train acc: 0.187000; val_acc: 0.181000
(Iteration 101 / 2450) loss: 1.840451
(Iteration 201 / 2450) loss: 1.744794
(Epoch 1 / 10) train acc: 0.383000; val_acc: 0.418000
(Iteration 301 / 2450) loss: 1.529679
(Iteration 401 / 2450) loss: 1.543941
(Epoch 2 / 10) train acc: 0.444000; val_acc: 0.439000
(Iteration 501 / 2450) loss: 1.552313
(Iteration 601 / 2450) loss: 1.511572
(Iteration 701 / 2450) loss: 1.462240
(Epoch 3 / 10) train acc: 0.476000; val_acc: 0.444000
(Iteration 801 / 2450) loss: 1.515084
(Iteration 901 / 2450) loss: 1.462425
(Epoch 4 / 10) train acc: 0.495000; val_acc: 0.469000
(Iteration 1001 / 2450) loss: 1.376394
(Iteration 1101 / 2450) loss: 1.612014
(Iteration 1201 / 2450) loss: 1.612220
(Epoch 5 / 10) train acc: 0.509000; val_acc: 0.477000
(Iteration 1301 / 2450) loss: 1.427094
(Iteration 1401 / 2450) loss: 1.422479
(Epoch 6 / 10) train acc: 0.535000; val_acc: 0.480000
(Iteration 1501 / 2450) loss: 1.395185
(Iteration 1601 / 2450) loss: 1.405752
(Iteration 1701 / 2450) loss: 1.210371
(Epoch 7 / 10) train acc: 0.543000; val_acc: 0.508000
(Iteration 1801 / 2450) loss: 1.204663
(Iteration 1901 / 2450) loss: 1.364992
(Epoch 8 / 10) train acc: 0.524000; val_acc: 0.488000
(Iteration 2001 / 2450) loss: 1.294265
(Iteration 2101 / 2450) loss: 1.311599
(Iteration 2201 / 2450) loss: 1.323174
(Epoch 9 / 10) train acc: 0.575000; val_acc: 0.509000
(Iteration 2301 / 2450) loss: 1.209796
(Iteration 2401 / 2450) loss: 1.320206
(Epoch 10 / 10) train acc: 0.528000; val_acc: 0.504000
```

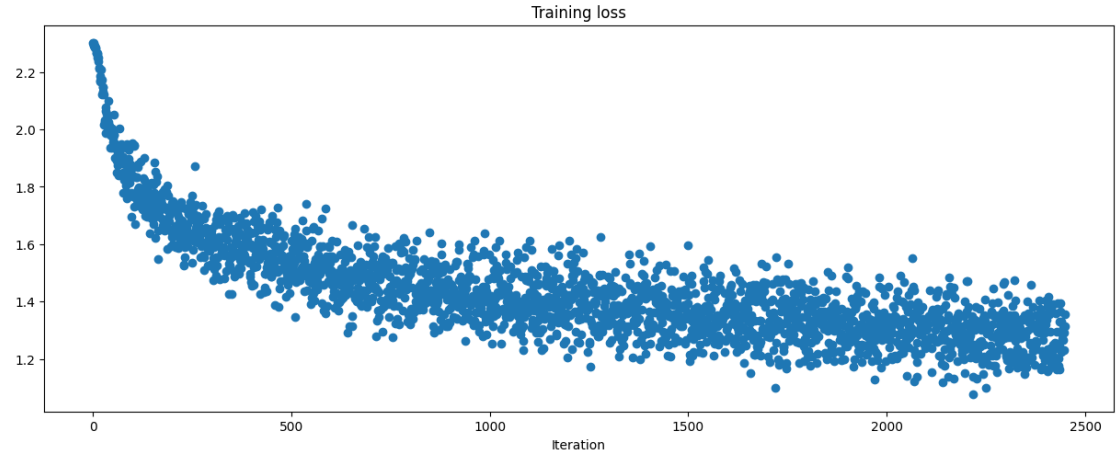
10 Debug the training

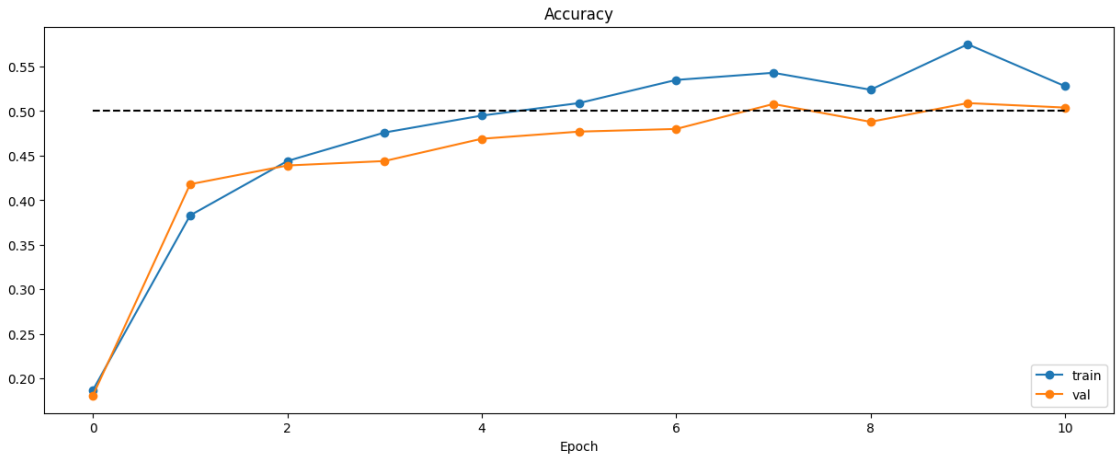
With the default parameters we provided above, you should get a validation accuracy of about 0.36 on the validation set. This isn't very good.

One strategy for getting insight into what's wrong is to plot the loss function and the accuracies on the training and validation sets during optimization.

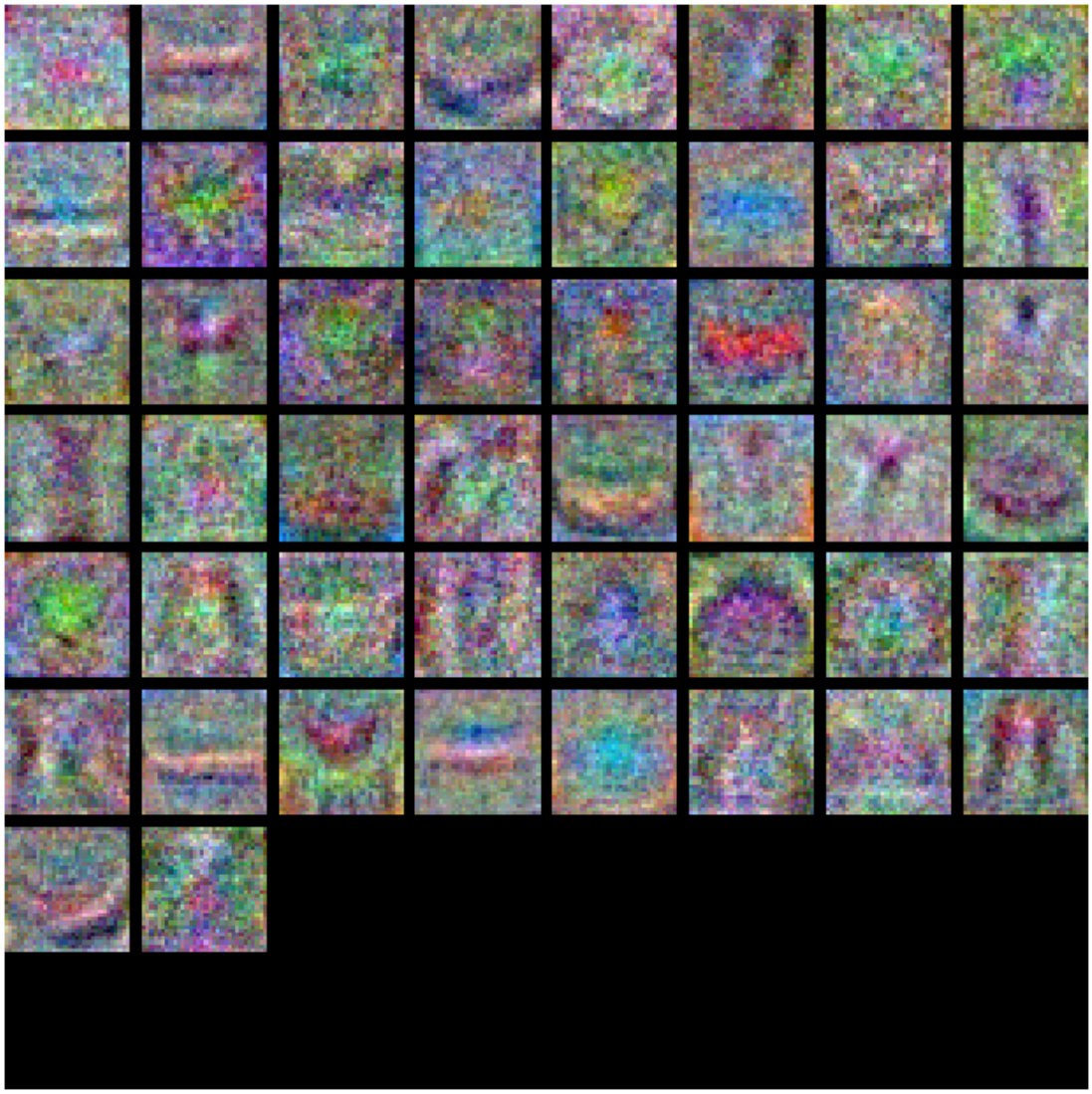
Another strategy is to visualize the weights that were learned in the first layer of the network. In most neural networks trained on visual data, the first layer weights typically show some visible structure when visualized.

[ ]:
```
# Run this cell to visualize training loss and train / val accuracy

plt.subplot(2, 1, 1)
plt.title('Training loss')
plt.plot(solver.loss_history, 'o')
plt.xlabel('Iteration')

plt.subplot(2, 1, 2)
plt.title('Accuracy')
plt.plot(solver.train_acc_history, '-o', label='train')
plt.plot(solver.val_acc_history, '-o', label='val')
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.gcf().set_size_inches(15, 12)
plt.show()
```





[ ]:
```python
from cs231n.vis_utils import visualize_grid

# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(3, 32, 32, -1).transpose(3, 1, 2, 0)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(model)
```



11 Tune your hyperparameters

What's wrong?. Looking at the visualizations above, we see that the loss is decreasing more or less linearly, which seems to suggest that the learning rate may be too low. Moreover, there is no gap between the training and validation accuracy, suggesting that the model we used has low capacity, and that we should increase its size. On the other hand, with a very large model we would expect to see more overfitting, which would manifest itself as a very large gap between the training and validation accuracy.

Tuning. Tuning the hyperparameters and developing intuition for how they affect the final performance is a large part of using Neural Networks, so we want you to get a lot of practice. Below, you should experiment with different values of the various hyperparameters, including hidden layer size, learning rate, numer of training epochs, and regularization strength. You might also consider

tuning the learning rate decay, but you should be able to get good performance using the default value.

Approximate results. You should be aim to achieve a classification accuracy of greater than 48% on the validation set. Our best network gets over 52% on the validation set.

Experiment: You goal in this exercise is to get as good of a result on CIFAR-10 as you can (52% could serve as a reference), with a fully-connected Neural Network. Feel free implement your own techniques (e.g. PCA to reduce dimensionality, or adding dropout, or adding features to the solver, etc.).

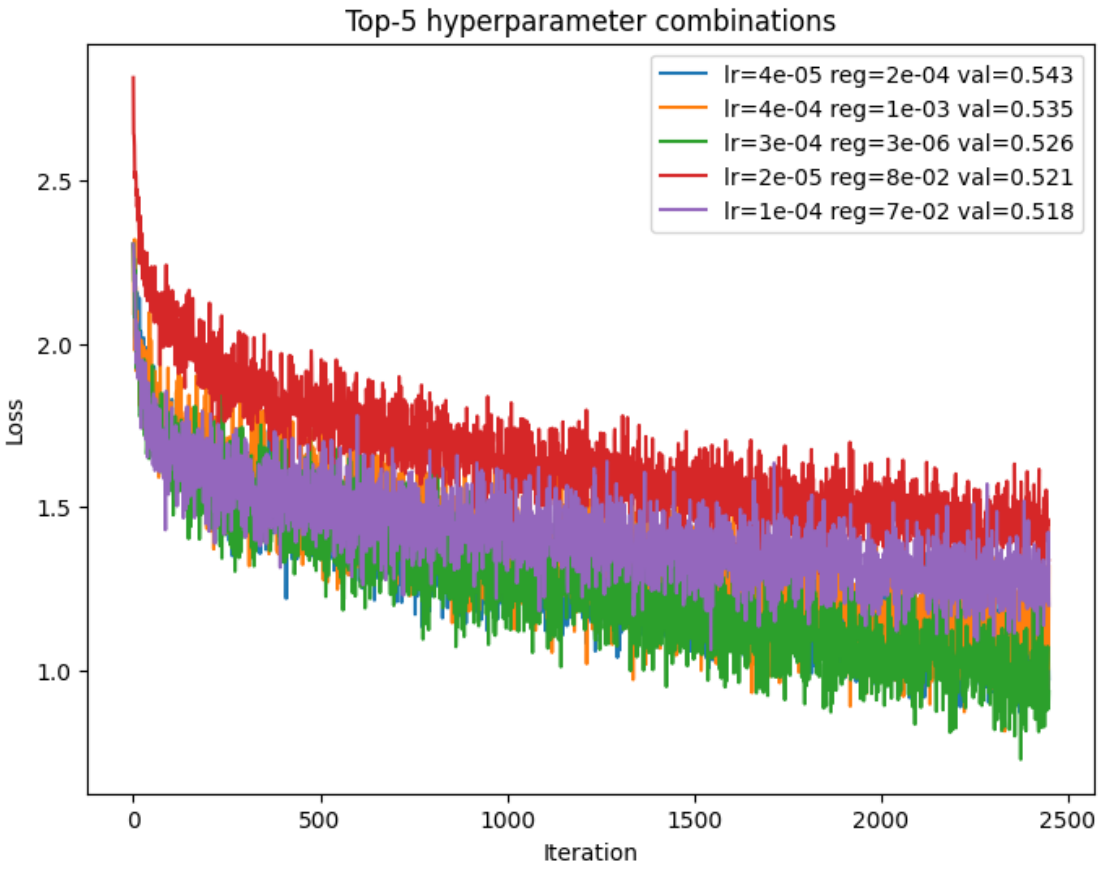
[ ]:
```
best_model = None

import numpy as np

################################################################################
#         learning_rate / reg / weight_scale / hidden_dims
################################################################################

np.random.seed(231)          #

NUM_TRIALS = 20              #          10     30+

# ---------- 1.      ----------
hidden_dims_pool = [
    [128],
    [256],
    [128, 128],
    [256, 256],
]

#
lr_samples  = 10 ** np.random.uniform(-5.0, -1.5, NUM_TRIALS)   #
reg_samples = 10 ** np.random.uniform(-6.0, -0.5, NUM_TRIALS)   # L2
ws_samples  = 10 ** np.random.uniform(-4.0, -1.5, NUM_TRIALS)   #

input_size  = 32 * 32 * 3
num_classes = 10

# ---------- 2.     ----------
results    = []      # (lr, reg, wsc, hidden_dims, train_acc, val_acc,␣
 ↪loss_hist, val_hist)
best_val   = -1

for trial in range(NUM_TRIALS):
    hidden_dims = hidden_dims_pool[np.random.randint(len(hidden_dims_pool))]
    lr  = lr_samples[trial]
    reg = reg_samples[trial]
```

```
    wsc = ws_samples[trial]

    model = FullyConnectedNet(
        hidden_dims,
        input_dim=input_size,
        num_classes=num_classes,
        reg=reg,
        weight_scale=wsc,
        dtype=np.float64,       #     np.float32
    )

    solver = Solver(
        model, data,
        update_rule='adam',                 # adam    sgd/sgd_momentum
        optim_config={'learning_rate': lr},
        lr_decay=0.95,
        num_epochs=10,
        batch_size=200,
        num_train_samples=2000,             #   epoch   2000
        print_every=100000,
        verbose=False,
    )
    solver.train()

    train_acc = solver.train_acc_history[-1]
    val_acc   = solver.val_acc_history[-1]
    results.append((lr, reg, wsc, hidden_dims, train_acc, val_acc,
                    solver.loss_history, solver.val_acc_history))

    if val_acc > best_val:
        best_val   = val_acc
        best_model = model

# ---------- 3.      val_acc     ----------
print('%-9s  %-9s  %-9s  %-14s  %-10s  %s' %
      ('lr', 'reg', 'w_scale', 'hidden_dims', 'train_acc', 'val_acc'))
for lr, reg, wsc, hd, ta, va, _, _ in sorted(results, key=lambda r: -r[5]):
    print('%-9.1e  %-9.1e  %-9.1e  %-14s  %-10.4f  %.4f'
          % (lr, reg, wsc, str(hd), ta, va))

print('\nbest validation accuracy achieved during cross-validation: %.4f' %␣
 ↪best_val)

# ---------- 4.    top-5    loss    ----------
plt.figure(figsize=(8, 6))
top5 = sorted(results, key=lambda r: -r[5])[:5]
for i, (lr, reg, wsc, hd, ta, va, loss_hist, _) in enumerate(top5):
```

```
    plt.plot(loss_hist, label='lr=%.0e reg=%.0e val=%.3f' % (lr, reg, va))
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.title('Top-5 hyperparameter combinations')
plt.legend()
plt.show()
```

lr	reg	w_scale	hidden_dims	train_acc	val_acc
4.3e-05	2.4e-04	1.3e-04	[256]	0.6390	0.5430
4.0e-04	1.0e-03	4.5e-04	[256]	0.6410	0.5350
3.1e-04	3.4e-06	2.5e-04	[256]	0.6590	0.5260
1.8e-05	7.5e-02	3.5e-03	[256]	0.5880	0.5210
1.3e-04	7.4e-02	1.4e-04	[256]	0.6380	0.5180
1.6e-05	9.2e-02	4.1e-03	[128]	0.5450	0.5150
1.4e-05	1.7e-06	4.7e-03	[128]	0.5595	0.5070
5.9e-05	2.5e-03	1.1e-04	[128, 128]	0.5360	0.5040
4.9e-04	1.3e-03	3.1e-02	[128, 128]	0.6305	0.4940
3.1e-05	1.6e-03	3.8e-04	[256, 256]	0.5275	0.4930
3.1e-04	1.7e-01	1.5e-04	[256]	0.5165	0.4900
1.7e-05	1.3e-03	6.3e-04	[256, 256]	0.4795	0.4890
1.1e-03	3.4e-05	1.0e-02	[256]	0.5405	0.4490
5.4e-03	1.4e-02	1.5e-02	[256]	0.4580	0.4290
1.2e-05	9.0e-02	4.5e-04	[128, 128]	0.3765	0.3930
1.5e-02	1.9e-02	4.1e-03	[128]	0.3875	0.3560
3.0e-03	2.7e-01	6.2e-03	[128, 128]	0.3445	0.3560
6.8e-03	2.1e-06	1.8e-03	[128, 128]	0.2010	0.2020
7.8e-03	2.3e-03	1.8e-02	[256, 256]	0.1050	0.1020
1.6e-02	2.1e-02	1.8e-04	[256, 256]	0.0960	0.0980

```
best validation accuracy achieved during cross-validation: 0.5430
```



[ ]:
```
print('   trial  :', len(results) if 'results' in dir() else 0)
print('  best_val:', best_val if 'best_val' in dir() else None)
```

```
  trial  : 20
best_val: 0.543
```

12 Test your model!

Run your best model on the validation and test sets. You should achieve above 48% accuracy on the validation set and the test set.

[ ]:
```
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
```

```
Validation set accuracy:  0.549
```

[ ]:
```
y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

```
Test set accuracy:  0.544
```

[ ]:
```
# Save best model
best_model.save("best_two_layer_net.npy")
```

```
best_two_layer_net.npy saved.
```

12.1 Inline Question 2:

Now that you have trained a Neural Network classifier, you may find that your testing accuracy is much lower than the training accuracy. In what ways can we decrease this gap? Select all that apply.

1. Train on a larger dataset.
2. Add more hidden units.
3. Increase the regularization strength.
4. None of the above.

YourAnswer :

1 and 3 (Train on a larger dataset; Increase the regularization strength).

YourExplanation :

The gap between training and test accuracy is the **generalization gap**, and its presence indicates the model is **overfitting**: it has enough capacity to memorize idiosyncrasies of the training set that do not transfer to unseen data. 1. **Train on a larger dataset — reduces the gap.** With more training examples, the model is forced to learn patterns that generalize rather than memorize; the training and validation distributions effectively overlap more. Empirically the learning curves of training and validation error converge as data size grows, so overfitting is directly mitigated. 2. **Add more hidden units — does *not* reduce the gap (it usually widens it).** Adding hidden units increases model capacity. A higher-capacity model can achieve even lower training loss by fitting noise in the training set, which typically makes the train/test discrepancy *larger*, not smaller. 3. **Increase the regularization strength — reduces the gap.** Regularization (e.g. the L2 penalty $\frac{\lambda}{2}\sum_w w^2$, dropout, etc.) explicitly penalizes complex solutions and constrains the effective capacity of the network. This trades a slightly higher training accuracy for better test accuracy, shrinking the gap. 4. Since options 1 and 3 are effective, "None of the above" is incorrect.

[ ]:
```

```

features

September 8, 2026

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/datasets
/content/drive/My Drive/cs231n/assignments/assignment1
```

1 Image features exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the assignments page on the course website.

We have seen that we can achieve reasonable performance on an image classification task by training a linear classifier on the pixels of the input image. In this exercise we will show that we can improve our classification performance by training linear classifiers not on raw pixels but on features that are computed from the raw pixels.

All of your work for this exercise will be done in this notebook.

[2]:
```
import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt


%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading extenrnal modules
# see http://stackoverflow.com/questions/1907993/
 ↪autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

1.1 Load data

Similar to previous exercises, we will load CIFAR-10 data from disk.

[3]:
```
from cs231n.features import color_histogram_hsv, hog_feature

def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000):
    # Load the raw CIFAR-10 data
    cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

    # Cleaning up variables to prevent loading data multiple times (which may␣
 ↪cause memory issue)
    try:
       del X_train, y_train
       del X_test, y_test
       print('Clear previously loaded data.')
    except:
       pass

    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    # Subsample the data
    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]
    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]
    mask = list(range(num_test))
    X_test = X_test[mask]
```

```
    y_test = y_test[mask]

    return X_train, y_train, X_val, y_val, X_test, y_test

X_train, y_train, X_val, y_val, X_test, y_test = get_CIFAR10_data()
```

1.2 Extract Features

For each image we will compute a Histogram of Oriented Gradients (HOG) as well as a color histogram using the hue channel in HSV color space. We form our final feature vector for each image by concatenating the HOG and color histogram feature vectors.

Roughly speaking, HOG should capture the texture of the image while ignoring color information, and the color histogram represents the color of the input image while ignoring texture. As a result, we expect that using both together ought to work better than using either alone. Verifying this assumption would be a good thing to try for your own interest.

The `hog_feature` and `color_histogram_hsv` functions both operate on a single image and return a feature vector for that image. The extract_features function takes a set of images and a list of feature functions and evaluates each feature function on each image, storing the results in a matrix where each column is the concatenation of all feature vectors for a single image.

[4]:
```
from cs231n.features import *

# num_color_bins = 10 # Number of bins in the color histogram
num_color_bins = 25 # Number of bins in the color histogram
feature_fns = [hog_feature, lambda img: color_histogram_hsv(img,␣
 ↪nbin=num_color_bins)]
X_train_feats = extract_features(X_train, feature_fns, verbose=True)
X_val_feats = extract_features(X_val, feature_fns)
X_test_feats = extract_features(X_test, feature_fns)

# Preprocessing: Subtract the mean feature
mean_feat = np.mean(X_train_feats, axis=0, keepdims=True)
X_train_feats -= mean_feat
X_val_feats -= mean_feat
X_test_feats -= mean_feat

# Preprocessing: Divide by standard deviation. This ensures that each feature
# has roughly the same scale.
std_feat = np.std(X_train_feats, axis=0, keepdims=True)
X_train_feats /= std_feat
X_val_feats /= std_feat
X_test_feats /= std_feat

# Preprocessing: Add a bias dimension
X_train_feats = np.hstack([X_train_feats, np.ones((X_train_feats.shape[0], 1))])
X_val_feats = np.hstack([X_val_feats, np.ones((X_val_feats.shape[0], 1))])
```

```
X_test_feats = np.hstack([X_test_feats, np.ones((X_test_feats.shape[0], 1))])
```

```
Done extracting features for 1000 / 49000 images
Done extracting features for 2000 / 49000 images
Done extracting features for 3000 / 49000 images
Done extracting features for 4000 / 49000 images
Done extracting features for 5000 / 49000 images
Done extracting features for 6000 / 49000 images
Done extracting features for 7000 / 49000 images
Done extracting features for 8000 / 49000 images
Done extracting features for 9000 / 49000 images
Done extracting features for 10000 / 49000 images
Done extracting features for 11000 / 49000 images
Done extracting features for 12000 / 49000 images
Done extracting features for 13000 / 49000 images
Done extracting features for 14000 / 49000 images
Done extracting features for 15000 / 49000 images
Done extracting features for 16000 / 49000 images
Done extracting features for 17000 / 49000 images
Done extracting features for 18000 / 49000 images
Done extracting features for 19000 / 49000 images
Done extracting features for 20000 / 49000 images
Done extracting features for 21000 / 49000 images
Done extracting features for 22000 / 49000 images
Done extracting features for 23000 / 49000 images
Done extracting features for 24000 / 49000 images
Done extracting features for 25000 / 49000 images
Done extracting features for 26000 / 49000 images
Done extracting features for 27000 / 49000 images
Done extracting features for 28000 / 49000 images
Done extracting features for 29000 / 49000 images
Done extracting features for 30000 / 49000 images
Done extracting features for 31000 / 49000 images
Done extracting features for 32000 / 49000 images
Done extracting features for 33000 / 49000 images
Done extracting features for 34000 / 49000 images
Done extracting features for 35000 / 49000 images
Done extracting features for 36000 / 49000 images
Done extracting features for 37000 / 49000 images
Done extracting features for 38000 / 49000 images
Done extracting features for 39000 / 49000 images
Done extracting features for 40000 / 49000 images
Done extracting features for 41000 / 49000 images
Done extracting features for 42000 / 49000 images
Done extracting features for 43000 / 49000 images
Done extracting features for 44000 / 49000 images
Done extracting features for 45000 / 49000 images
```

```
Done extracting features for 46000 / 49000 images
Done extracting features for 47000 / 49000 images
Done extracting features for 48000 / 49000 images
Done extracting features for 49000 / 49000 images
```

1.3 Train Softmax classifier on features

Using the Softmax code developed earlier in the assignment, train Softmax classifiers on top of the features extracted above; this should achieve better results than training them directly on top of raw pixels.

[5]:
```
# Use the validation set to tune the learning rate and regularization strength

from cs231n.classifiers.linear_classifier import Softmax

# Expanded hyperparameter search space
learning_rates = [1e-7, 5e-7, 1e-6]
regularization_strengths = [5e4, 1e5, 5e5, 1e6, 5e6]

results = {}
best_val = -1
best_softmax = None

################################################################################
# TODO:                                                                        #
# Use the validation set to set the learning rate and regularization strength. #
# This should be identical to the validation that you did for the Softmax; save#
# the best trained classifer in best_softmax. If you carefully tune the model, #
# you should be able to get accuracy of above 0.42 on the validation set.      #
################################################################################

for lr in learning_rates:
    for reg in regularization_strengths:
        softmax = Softmax()
        softmax.train(X_train_feats, y_train, learning_rate=lr, reg=reg,
                      num_iters=1500, verbose=False)

        # Evaluate on training set
        y_train_pred = softmax.predict(X_train_feats)
        training_accuracy = np.mean(y_train == y_train_pred)

        # Evaluate on validation set
        y_val_pred = softmax.predict(X_val_feats)
        val_accuracy = np.mean(y_val == y_val_pred)

        # Store results
        results[(lr, reg)] = (training_accuracy, val_accuracy)
```

```
        # Track best classifier
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_softmax = softmax
################################################################################
#                              END OF YOUR CODE                                #
################################################################################

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
                lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved: %f' % best_val)
```

```
/content/drive/My
Drive/cs231n/assignments/assignment1/cs231n/classifiers/softmax.py:89:
RuntimeWarning: divide by zero encountered in log
  correct_logprobs = -np.log(p[np.arange(num_train), y])   # (N,)
/content/drive/My
Drive/cs231n/assignments/assignment1/cs231n/classifiers/softmax.py:91:
RuntimeWarning: overflow encountered in scalar multiply
  loss += reg * np.sum(W * W)
/usr/local/lib/python3.11/dist-packages/numpy/_core/fromnumeric.py:86:
RuntimeWarning: overflow encountered in reduce
  return ufunc.reduce(obj, axis, dtype, out, **passkwargs)
/content/drive/My
Drive/cs231n/assignments/assignment1/cs231n/classifiers/softmax.py:91:
RuntimeWarning: overflow encountered in multiply
  loss += reg * np.sum(W * W)
/content/drive/My
Drive/cs231n/assignments/assignment1/cs231n/classifiers/softmax.py:108:
RuntimeWarning: overflow encountered in multiply
  dW = dW / num_train + 2 * reg * W
```

```
lr 1.000000e-07 reg 5.000000e+04 train accuracy: 0.422327 val accuracy: 0.431000
lr 1.000000e-07 reg 1.000000e+05 train accuracy: 0.423408 val accuracy: 0.419000
lr 1.000000e-07 reg 5.000000e+05 train accuracy: 0.414551 val accuracy: 0.415000
lr 1.000000e-07 reg 1.000000e+06 train accuracy: 0.402837 val accuracy: 0.418000
lr 1.000000e-07 reg 5.000000e+06 train accuracy: 0.350184 val accuracy: 0.369000
lr 5.000000e-07 reg 5.000000e+04 train accuracy: 0.417204 val accuracy: 0.416000
lr 5.000000e-07 reg 1.000000e+05 train accuracy: 0.411143 val accuracy: 0.402000
lr 5.000000e-07 reg 5.000000e+05 train accuracy: 0.370204 val accuracy: 0.388000
lr 5.000000e-07 reg 1.000000e+06 train accuracy: 0.337082 val accuracy: 0.341000
lr 5.000000e-07 reg 5.000000e+06 train accuracy: 0.100265 val accuracy: 0.087000
lr 1.000000e-06 reg 5.000000e+04 train accuracy: 0.420918 val accuracy: 0.420000
lr 1.000000e-06 reg 1.000000e+05 train accuracy: 0.414673 val accuracy: 0.423000
```

```
lr 1.000000e-06 reg 5.000000e+05 train accuracy: 0.324837 val accuracy: 0.335000
lr 1.000000e-06 reg 1.000000e+06 train accuracy: 0.107000 val accuracy: 0.115000
lr 1.000000e-06 reg 5.000000e+06 train accuracy: 0.100265 val accuracy: 0.087000
best validation accuracy achieved: 0.431000
```

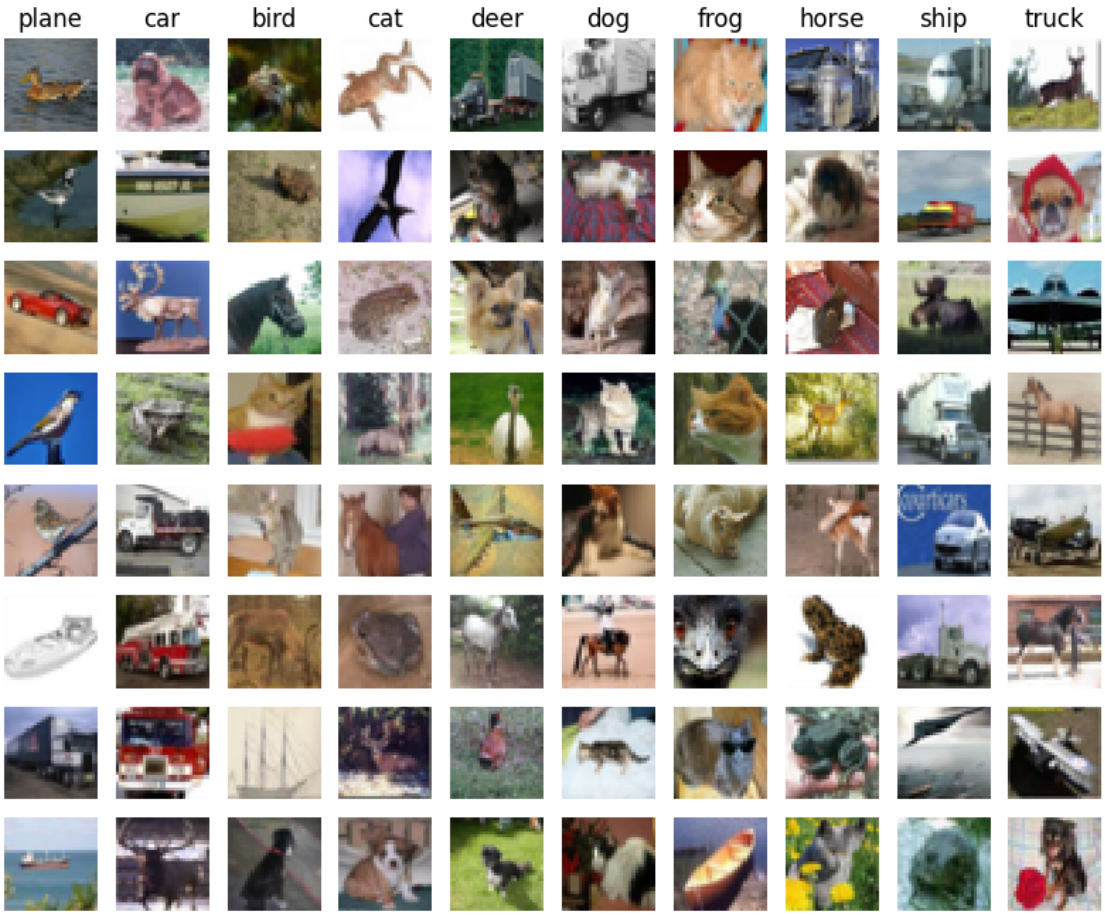
[6]:
```
# Evaluate your trained Softmax on the test set: you should be able to get at␣
 ↪least 0.42
y_test_pred = best_softmax.predict(X_test_feats)
test_accuracy = np.mean(y_test == y_test_pred)
print(test_accuracy)
```

```
0.423
```

[7]:
```
# Save best softmax model
best_softmax.save("best_softmax_features.npy")
```

```
best_softmax_features.npy saved.
```

[8]:
```
# An important way to gain intuition about how an algorithm works is to
# visualize the mistakes that it makes. In this visualization, we show examples
# of images that are misclassified by our current system. The first column
# shows images that our system labeled as "plane" but whose true label is
# something other than "plane".

examples_per_class = 8
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
 ↪'ship', 'truck']
for cls, cls_name in enumerate(classes):
    idxs = np.where((y_test != cls) & (y_test_pred == cls))[0]
    idxs = np.random.choice(idxs, examples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt.subplot(examples_per_class, len(classes), i * len(classes) + cls +␣
 ↪1)
        plt.imshow(X_test[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls_name)
plt.show()
```



1.3.1 Inline question 1:

Describe the misclassification results that you see. Do they make sense?

YourAnswer :

The misclassified images show clear structure rather than random errors, and the confusions make sense given the features we used. 1. **Similar shapes get confused.** The most common confusions are between semantically similar categories: car vs. truck, cat vs. dog, deer vs. horse. These pairs share boxy silhouettes (vehicles) or four-legged body outlines (animals), so their HOG edge-orientation histograms are very similar. 2. **Backgrounds dominate the color histogram.** Many images predicted as "plane" are actually birds or ships, because all three categories are typically photographed against large regions of blue sky or sea. Since the HSV color histogram only captures the global color distribution, these images have nearly identical color features even though the objects differ. 3. **Color-matched categories leak into each other.** Images with green backgrounds (frogs, deer in forests) are frequently mislabeled as each other for the same reason. **Why this happens:** our feature vector is $\phi(x) = [\phi_{\text{HOG}}(x), \phi_{\text{color}}(x)]$. This representation preserves two kinds of information: edge-orientation statistics (rough shape cues) and global color distributions. It discards fine spatial layout and texture detail. As a result, two images are easily

confused whenever their distance in feature space is small:

$$\|\phi(x_i) - \phi(x_j)\| \text{ small } \iff \text{similar edge statistics or similar color palettes,}$$

even if the actual objects are semantically different. HOG and color histograms are purely low-level cues with no object-level reasoning, so failures like bird classified as plane (shared blue background) or cat classified as dog (shared texture and body plan) are exactly what this representation predicts. This also explains why the two-layer neural network on these same features tops out around 58-60% accuracy: the bottleneck is the expressiveness of the hand-crafted features, not the classifier itself. A more powerful classifier cannot recover information (spatial arrangement, fine texture) that was already discarded by the feature extraction step.

1.4 Neural Network on image features

Earlier in this assigment we saw that training a two-layer neural network on raw pixels achieved better classification performance than linear classifiers on raw pixels. In this notebook we have seen that linear classifiers on image features outperform linear classifiers on raw pixels.

For completeness, we should also try training a neural network on image features. This approach should outperform all previous approaches: you should easily be able to achieve over 55% classification accuracy on the test set; our best model achieves about 60% classification accuracy.

[9]:
```
# Preprocessing: Remove the bias dimension
# Make sure to run this cell only ONCE
print(X_train_feats.shape)
X_train_feats = X_train_feats[:, :-1]
X_val_feats = X_val_feats[:, :-1]
X_test_feats = X_test_feats[:, :-1]

print(X_train_feats.shape)
```

```
(49000, 170)
(49000, 169)
```

[11]:
```
from cs231n.classifiers.fc_net import TwoLayerNet
from cs231n.solver import Solver

input_dim = X_train_feats.shape[1]
hidden_dim = 500
num_classes = 10

data = {
    'X_train': X_train_feats,
    'y_train': y_train,
    'X_val': X_val_feats,
    'y_val': y_val,
    'X_test': X_test_feats,
    'y_test': y_test,
}
```

```
net = TwoLayerNet(input_dim, hidden_dim, num_classes)
best_net = None

################################################################################
# TODO: Train a two-layer neural network on image features. You may want to    #
# cross-validate various parameters as in previous sections. Store your best   #
# model in the best_net variable.                                              #
################################################################################

best_val = -1
best_net = None
best_params = None

# Adam
learning_rates = [1e-4, 5e-4, 1e-3, 5e-3]
regularization_strengths = [1e-5, 1e-4, 1e-3]

for lr in learning_rates:
    for reg in regularization_strengths:
        print('lr=%e reg=%e' % (lr, reg))

        net = TwoLayerNet(input_dim, hidden_dim, num_classes, reg=reg)

        solver = Solver(net, data,
                        update_rule='adam',
                        optim_config={'learning_rate': lr},
                        lr_decay=0.95,
                        num_epochs=15,
                        batch_size=256,
                        verbose=False)
        solver.train()

        print('    best val acc: %f' % solver.best_val_acc)

        if solver.best_val_acc > best_val:
            best_val = solver.best_val_acc
            best_net = net
            best_params = (lr, reg)

print('best lr=%e, best reg=%e' % best_params)
print('best validation accuracy achieved: %f' % best_val)

################################################################################
#                              END OF YOUR CODE                                #
################################################################################
```

```
lr=1.000000e-04 reg=1.000000e-05
```

```
    best val acc: 0.539000
lr=1.000000e-04 reg=1.000000e-04
    best val acc: 0.543000
lr=1.000000e-04 reg=1.000000e-03
    best val acc: 0.543000
lr=5.000000e-04 reg=1.000000e-05
    best val acc: 0.626000
lr=5.000000e-04 reg=1.000000e-04
    best val acc: 0.616000
lr=5.000000e-04 reg=1.000000e-03
    best val acc: 0.599000
lr=1.000000e-03 reg=1.000000e-05
    best val acc: 0.609000
lr=1.000000e-03 reg=1.000000e-04
    best val acc: 0.609000
lr=1.000000e-03 reg=1.000000e-03
    best val acc: 0.625000
lr=5.000000e-03 reg=1.000000e-05
    best val acc: 0.579000
lr=5.000000e-03 reg=1.000000e-04
    best val acc: 0.590000
lr=5.000000e-03 reg=1.000000e-03
    best val acc: 0.609000
best lr=5.000000e-04, best reg=1.000000e-05
best validation accuracy achieved: 0.626000
```

[12]:
```
# Run your best neural net classifier on the test set. You should be able
# to get more than 58% accuracy. It is also possible to get >60% accuracy
# with careful tuning.

y_test_pred = np.argmax(best_net.loss(data['X_test']), axis=1)
test_acc = (y_test_pred == data['y_test']).mean()
print(test_acc)
```

```
0.595
```

[13]:
```
# Save best model
best_net.save("best_two_layer_net_features.npy")
```

```
best_two_layer_net_features.npy saved.
```

[ ]:
```

```

FullyConnectedNets

September 8, 2026

[2]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
# %cd /content/drive/My\ Drive/$FOLDERNAME
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/datasets
/content/drive/My Drive/cs231n/assignments/assignment1
```

1 Multi-Layer Fully Connected Network

In this exercise, you will implement a fully connected network with an arbitrary number of hidden layers.

[3]:
```
# from google.colab import drive
# drive.mount('/content/drive')
```

Read through the `FullyConnectedNet` class in the file `cs231n/classifiers/fc_net.py`.

Implement the network initialization, forward pass, and backward pass. Throughout this assignment, you will be implementing layers in `cs231n/layers.py`. You can re-use your implementations

for `affine_forward`, `affine_backward`, `relu_forward`, `relu_backward`, and `softmax_loss` from before. For right now, don't worry about implementing dropout or batch/layer normalization yet, as you will add those features later.

[4]:
```
# Setup cell.
import time
import numpy as np
import matplotlib.pyplot as plt
from cs231n.classifiers.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient,␣
 ↪eval_numerical_gradient_array
from cs231n.solver import Solver

%matplotlib inline
plt.rcParams["figure.figsize"] = (10.0, 8.0)  # Set default size of plots.
plt.rcParams["image.interpolation"] = "nearest"
plt.rcParams["image.cmap"] = "gray"

%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """Returns relative error."""
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

[5]:
```
# Load the (preprocessed) CIFAR-10 data.
data = get_CIFAR10_data()
for k, v in list(data.items()):
    print(f"{k}: {v.shape}")
```

```
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

1.1 Initial Loss and Gradient Check

As a sanity check, run the following to check the initial loss and to gradient check the network both with and without regularization. This is a good way to see if the initial losses seem reasonable.

For gradient checking, you should expect to see errors around 1e-7 or less.

[6]:
```
np.random.seed(231)
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))
```

```
for reg in [0, 3.14]:
    print("Running check with reg = ", reg)
    model = FullyConnectedNet(
        [H1, H2],
        input_dim=D,
        num_classes=C,
        reg=reg,
        weight_scale=5e-2,
        dtype=np.float64
    )

    loss, grads = model.loss(X, y)
    print("Initial loss: ", loss)

    # Most of the errors should be on the order of e-7 or smaller.
    # NOTE: It is fine however to see an error for W2 on the order of e-5
    # for the check when reg = 0.0
    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name],␣
 ↪verbose=False, h=1e-5)
        print(f"{name} relative error: {rel_error(grad_num, grads[name])}")
```

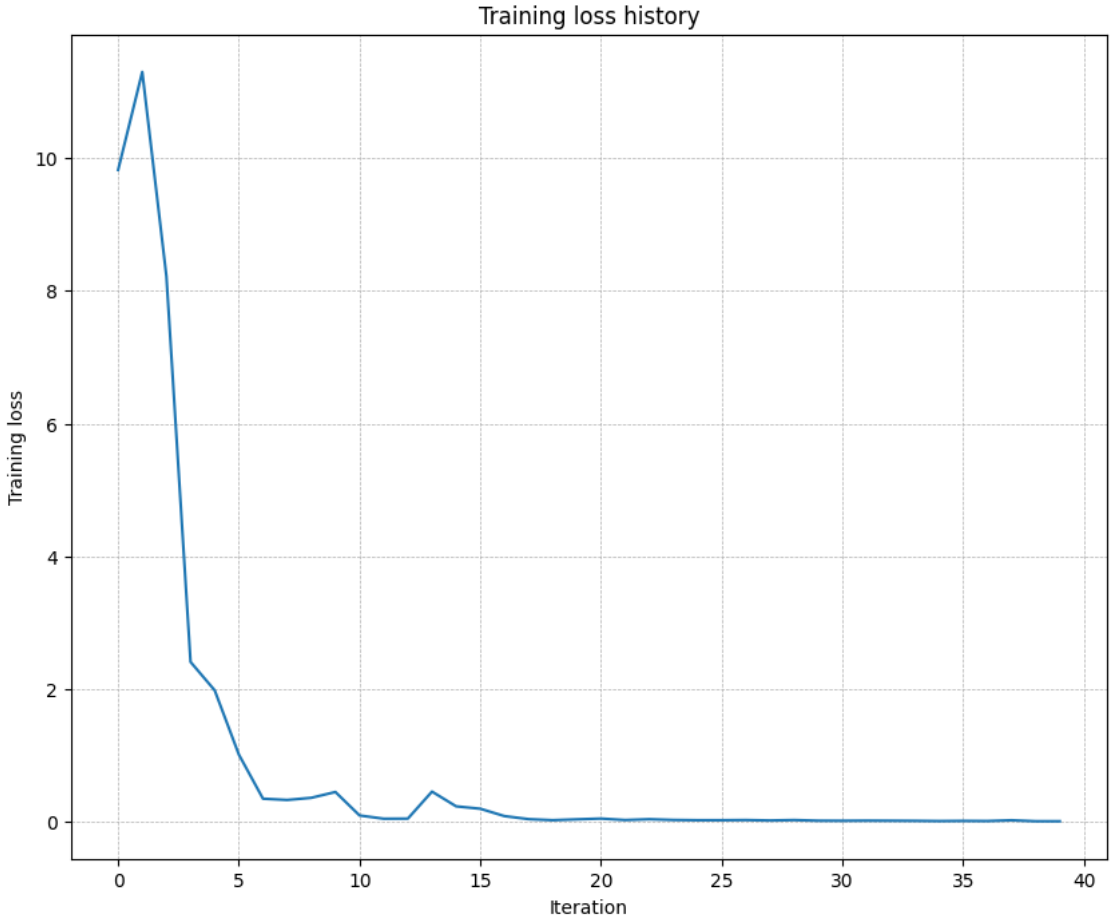
```
Running check with reg =  0
Initial loss:  2.3004790897684924
W1 relative error: 1.4839894098713283e-07
W2 relative error: 2.21204793107852e-05
W3 relative error: 3.527252851540647e-07
b1 relative error: 5.376386228531692e-09
b2 relative error: 2.085654200257447e-09
b3 relative error: 5.7957243458479405e-11
Running check with reg =  3.14
Initial loss:  7.052114776533016
W1 relative error: 3.904541941902138e-09
W2 relative error: 6.86942277940646e-08
W3 relative error: 2.131129848945024e-08
b1 relative error: 1.4752428222134868e-08
b2 relative error: 1.7223750761525226e-09
b3 relative error: 1.5702714832602802e-10
```

As another sanity check, make sure your network can overfit on a small dataset of 50 images. First, we will try a three-layer network with 100 units in each hidden layer. In the following cell, tweak the **learning rate** and **weight initialization scale** to overfit and achieve 100% training accuracy within 20 epochs.

[7]:
```
# TODO: Use a three-layer Net to overfit 50 training examples by
# tweaking just the learning rate and initialization scale.

num_train = 50
small_data = {
  "X_train": data["X_train"][:num_train],
  "y_train": data["y_train"][:num_train],
  "X_val": data["X_val"],
  "y_val": data["y_val"],
}

weight_scale = 3e-2   # Experiment with this!
learning_rate = 4e-3  # Experiment with this!


model = FullyConnectedNet(
    [100, 100],
    weight_scale=weight_scale,
    dtype=np.float64
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule="sgd",
    optim_config={"learning_rate": learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title("Training loss history")
plt.xlabel("Iteration")
plt.ylabel("Training loss")
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
(Iteration 1 / 40) loss: 9.817983
(Epoch 0 / 20) train acc: 0.200000; val_acc: 0.116000
(Epoch 1 / 20) train acc: 0.400000; val_acc: 0.132000
(Epoch 2 / 20) train acc: 0.640000; val_acc: 0.133000
(Epoch 3 / 20) train acc: 0.860000; val_acc: 0.148000
(Epoch 4 / 20) train acc: 0.900000; val_acc: 0.163000
(Epoch 5 / 20) train acc: 0.960000; val_acc: 0.165000
(Iteration 11 / 40) loss: 0.099653
(Epoch 6 / 20) train acc: 0.960000; val_acc: 0.164000
```

```
(Epoch 7 / 20) train acc: 0.960000; val_acc: 0.173000
(Epoch 8 / 20) train acc: 1.000000; val_acc: 0.167000
(Epoch 9 / 20) train acc: 1.000000; val_acc: 0.165000
(Epoch 10 / 20) train acc: 1.000000; val_acc: 0.167000
(Iteration 21 / 40) loss: 0.052787
(Epoch 11 / 20) train acc: 1.000000; val_acc: 0.164000
(Epoch 12 / 20) train acc: 1.000000; val_acc: 0.166000
(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.166000
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.171000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.166000
(Iteration 31 / 40) loss: 0.019841
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.169000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.170000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.169000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.173000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.171000
```



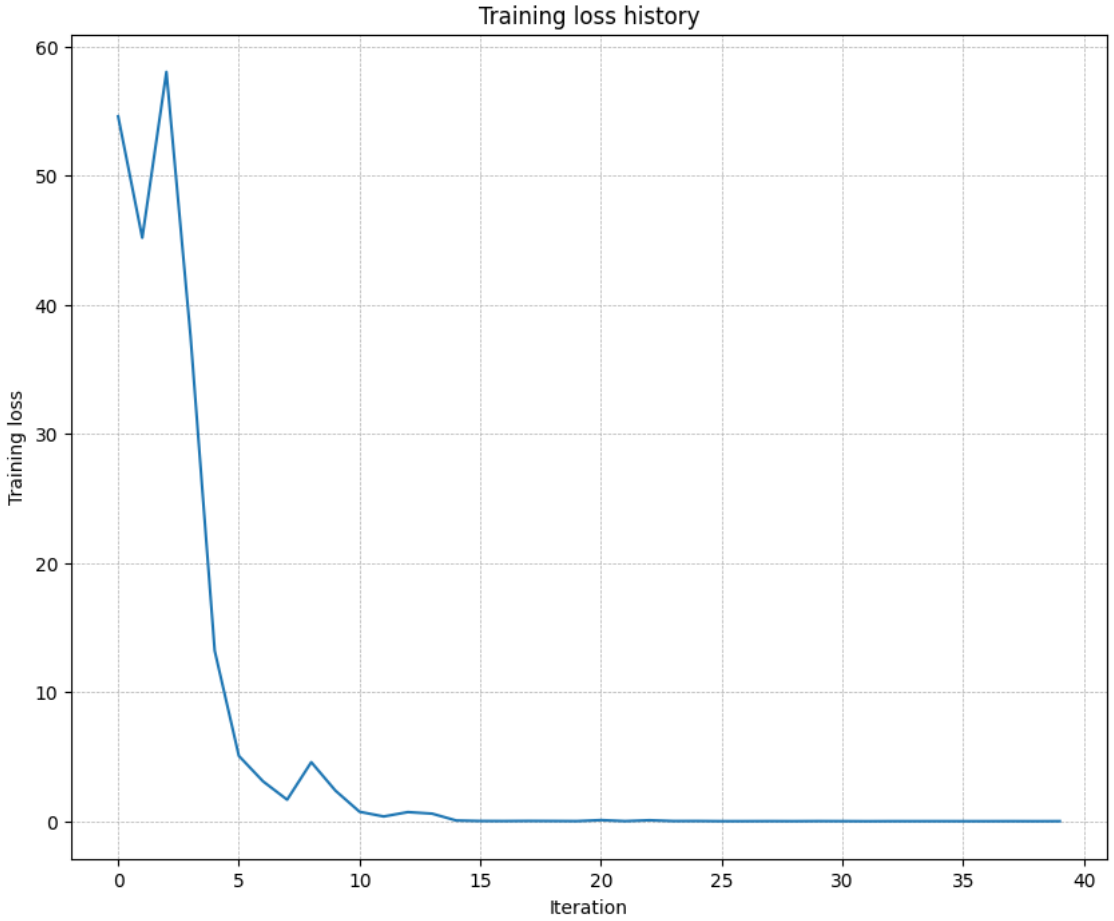
Now, try to use a five-layer network with 100 units on each layer to overfit on 50 training examples. Again, you will have to adjust the learning rate and weight initialization scale, but you should be

able to achieve 100% training accuracy within 20 epochs.

[8]:
```
# TODO: Use a five-layer Net to overfit 50 training examples by
# tweaking just the learning rate and initialization scale.

num_train = 50
small_data = {
  'X_train': data['X_train'][:num_train],
  'y_train': data['y_train'][:num_train],
  'X_val': data['X_val'],
  'y_val': data['y_val'],
}

learning_rate = 3e-3  # Experiment with this!
weight_scale = 8e-2   # Experiment with this!


model = FullyConnectedNet(
    [100, 100, 100, 100],
    weight_scale=weight_scale,
    dtype=np.float64
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule='sgd',
    optim_config={'learning_rate': learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
(Iteration 1 / 40) loss: 54.581332
(Epoch 0 / 20) train acc: 0.160000; val_acc: 0.111000
(Epoch 1 / 20) train acc: 0.260000; val_acc: 0.087000
(Epoch 2 / 20) train acc: 0.280000; val_acc: 0.115000
(Epoch 3 / 20) train acc: 0.540000; val_acc: 0.130000
(Epoch 4 / 20) train acc: 0.700000; val_acc: 0.133000
(Epoch 5 / 20) train acc: 0.800000; val_acc: 0.117000
(Iteration 11 / 40) loss: 0.737084
```

```
(Epoch 6 / 20) train acc: 0.880000; val_acc: 0.129000
(Epoch 7 / 20) train acc: 0.960000; val_acc: 0.138000
(Epoch 8 / 20) train acc: 0.960000; val_acc: 0.135000
(Epoch 9 / 20) train acc: 0.960000; val_acc: 0.134000
(Epoch 10 / 20) train acc: 0.960000; val_acc: 0.131000
(Iteration 21 / 40) loss: 0.092505
(Epoch 11 / 20) train acc: 0.980000; val_acc: 0.132000
(Epoch 12 / 20) train acc: 1.000000; val_acc: 0.129000
(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.130000
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.129000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.129000
(Iteration 31 / 40) loss: 0.010478
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.129000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.130000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.129000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.127000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.131000
```



1.2 Inline Question 1:

Did you notice anything about the comparative difficulty of training the three-layer network vs. training the five-layer network? In particular, based on your experience, which network seemed more sensitive to the initialization scale? Why do you think that is the case?

1.3 Answer:

Yes — the five-layer network was far more sensitive to the initialization scale than the three-layer network. The three-layer net could overfit the 50 training examples with a fairly wide range of (weight_scale, learning_rate) combinations, but when the five-layer net was initialized with weight_scale = 1e-5 the loss barely moved at all; we had to increase the scale by roughly an order of magnitude (e.g. ~8e-2) before training made progress.

The reason is that depth compounds the effect of initialization multiplicatively. In the forward pass, activations are repeatedly multiplied by the weight matrices, so with small weights the activation magnitude decays roughly like w^L through L layers (and ReLU compounds this by zeroing out about half the units at each layer). Symmetrically, in the backward pass each gradient must be multiplied by the same small weight matrices on the way back, so gradients arriving at the early layers shrink exponentially with depth — those layers receive almost no learning signal and barely change. With weights that are too large the opposite happens and activations/gradients explode.

Because these effects accumulate with every extra layer, the range of initialization scales that keeps signals healthy — the "sweet spot" — shrinks dramatically as the network gets deeper. This is exactly the motivation for principled initialization schemes such as Xavier/He initialization, which set the weight variance as ~1/fan_in (or 2/fan_in for ReLU) so that signal variance stays roughly constant regardless of depth.

2 Update rules

So far we have used vanilla stochastic gradient descent (SGD) as our update rule. More sophisticated update rules can make it easier to train deep networks. We will implement a few of the most commonly used update rules and compare them to vanilla SGD.

2.1 SGD+Momentum

Stochastic gradient descent with momentum is a widely used update rule that tends to make deep networks converge faster than vanilla stochastic gradient descent. See the Momentum Update section at http://cs231n.github.io/neural-networks-3/#sgd for more information.

Open the file `cs231n/optim.py` and read the documentation at the top of the file to make sure you understand the API. Implement the SGD+momentum update rule in the function `sgd_momentum` and run the following to check your implementation. You should see errors less than e-8.

[9]:
```
from cs231n.optim import sgd_momentum

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
v = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)
```

```
config = {"learning_rate": 1e-3, "velocity": v}
next_w, _ = sgd_momentum(w, dw, config=config)

expected_next_w = np.asarray([
  [ 0.1406,      0.20738947,  0.27417895,  0.34096842,  0.40775789],
  [ 0.47454737,  0.54133684,  0.60812632,  0.67491579,  0.74170526],
  [ 0.80849474,  0.87528421,  0.94207368,  1.00886316,  1.07565263],
  [ 1.14244211,  1.20923158,  1.27602105,  1.34281053,  1.4096    ]])
expected_velocity = np.asarray([
  [ 0.5406,      0.55475789,  0.56891579, 0.58307368,  0.59723158],
  [ 0.61138947,  0.62554737,  0.63970526,  0.65386316,  0.66802105],
  [ 0.68217895,  0.69633684,  0.71049474,  0.72465263,  0.73881053],
  [ 0.75296842,  0.76712632,  0.78128421,  0.79544211,  0.8096    ]])

# Should see relative errors around e-8 or less
print("next_w error: ", rel_error(next_w, expected_next_w))
print("velocity error: ", rel_error(expected_velocity, config["velocity"]))
```

```
next_w error:  8.882347033505819e-09
velocity error:  4.269287743278663e-09
```

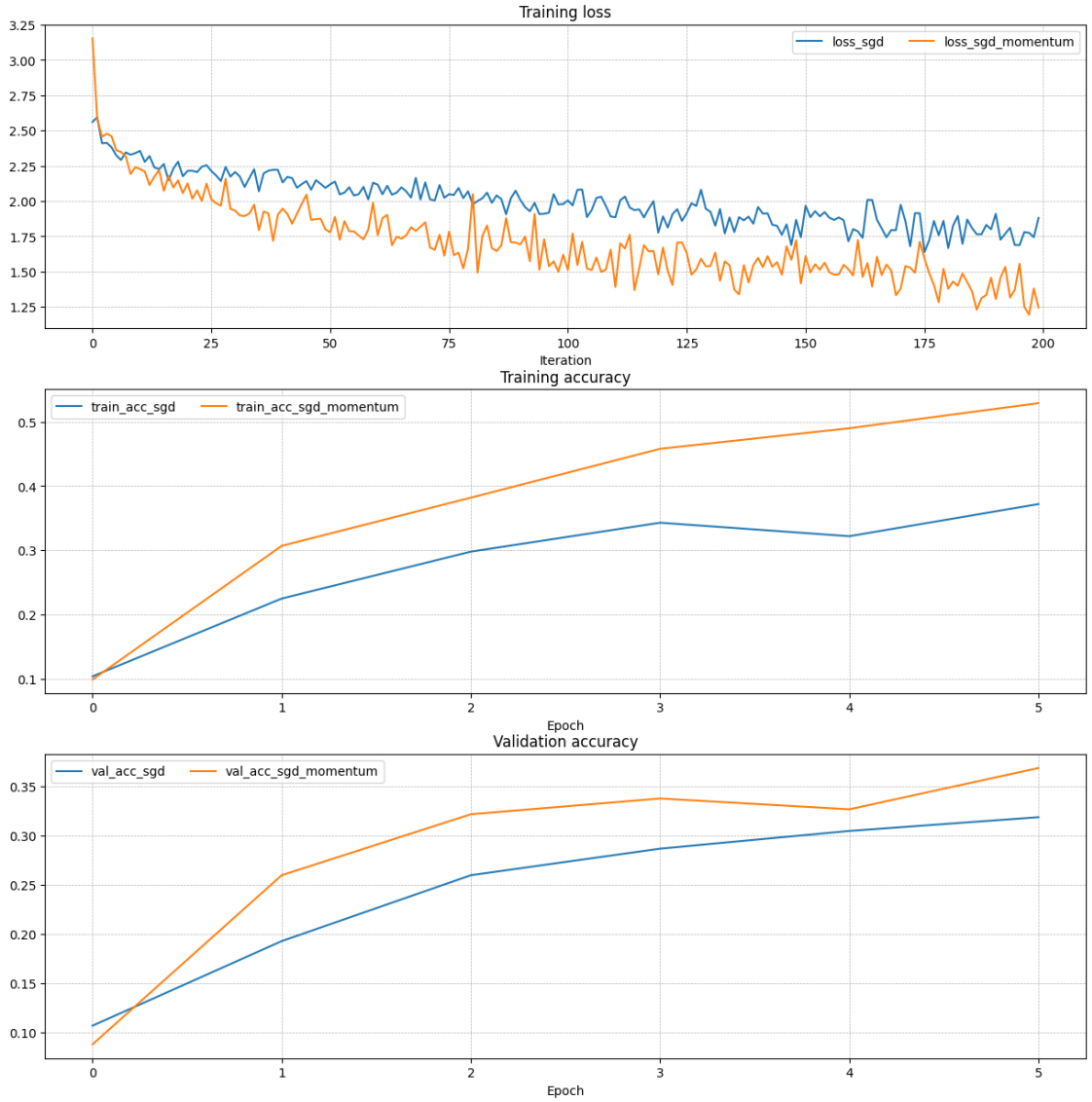
Once you have done so, run the following to train a six-layer network with both SGD and SGD+momentum. You should see the SGD+momentum update rule converge faster.

[10]:
```
num_train = 4000
small_data = {
  'X_train': data['X_train'][:num_train],
  'y_train': data['y_train'][:num_train],
  'X_val': data['X_val'],
  'y_val': data['y_val'],
}

solvers = {}

for update_rule in ['sgd', 'sgd_momentum']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )

    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
```

```
        optim_config={'learning_rate': 5e-3},
        verbose=True,
    )
    solvers[update_rule] = solver
    solver.train()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"loss_{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"train_acc_{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"val_acc_{update_rule}")

for ax in axes:
    ax.legend(loc="best", ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()
```

```
Running with  sgd
(Iteration 1 / 200) loss: 2.559978
(Epoch 0 / 5) train acc: 0.104000; val_acc: 0.107000
(Iteration 11 / 200) loss: 2.356070
(Iteration 21 / 200) loss: 2.214091
(Iteration 31 / 200) loss: 2.205928
(Epoch 1 / 5) train acc: 0.225000; val_acc: 0.193000
(Iteration 41 / 200) loss: 2.132095
(Iteration 51 / 200) loss: 2.118950
(Iteration 61 / 200) loss: 2.116443
(Iteration 71 / 200) loss: 2.132549
(Epoch 2 / 5) train acc: 0.298000; val_acc: 0.260000
(Iteration 81 / 200) loss: 1.977227
(Iteration 91 / 200) loss: 2.007528
(Iteration 101 / 200) loss: 2.004762
(Iteration 111 / 200) loss: 1.885342
(Epoch 3 / 5) train acc: 0.343000; val_acc: 0.287000
(Iteration 121 / 200) loss: 1.891517
(Iteration 131 / 200) loss: 1.923677
(Iteration 141 / 200) loss: 1.957743
(Iteration 151 / 200) loss: 1.966736
```

```
(Epoch 4 / 5) train acc: 0.322000; val_acc: 0.305000
(Iteration 161 / 200) loss: 1.801483
(Iteration 171 / 200) loss: 1.973780
(Iteration 181 / 200) loss: 1.666572
(Iteration 191 / 200) loss: 1.909494
(Epoch 5 / 5) train acc: 0.372000; val_acc: 0.319000
Running with  sgd_momentum
(Iteration 1 / 200) loss: 3.153778
(Epoch 0 / 5) train acc: 0.099000; val_acc: 0.088000
(Iteration 11 / 200) loss: 2.227203
(Iteration 21 / 200) loss: 2.125706
(Iteration 31 / 200) loss: 1.932695
(Epoch 1 / 5) train acc: 0.307000; val_acc: 0.260000
(Iteration 41 / 200) loss: 1.946488
(Iteration 51 / 200) loss: 1.778584
(Iteration 61 / 200) loss: 1.758119
(Iteration 71 / 200) loss: 1.849137
(Epoch 2 / 5) train acc: 0.382000; val_acc: 0.322000
(Iteration 81 / 200) loss: 2.048671
(Iteration 91 / 200) loss: 1.693223
(Iteration 101 / 200) loss: 1.511693
(Iteration 111 / 200) loss: 1.390754
(Epoch 3 / 5) train acc: 0.458000; val_acc: 0.338000
(Iteration 121 / 200) loss: 1.670614
(Iteration 131 / 200) loss: 1.540271
(Iteration 141 / 200) loss: 1.597365
(Iteration 151 / 200) loss: 1.609851
(Epoch 4 / 5) train acc: 0.490000; val_acc: 0.327000
(Iteration 161 / 200) loss: 1.472687
(Iteration 171 / 200) loss: 1.378620
(Iteration 181 / 200) loss: 1.378175
(Iteration 191 / 200) loss: 1.306439
(Epoch 5 / 5) train acc: 0.529000; val_acc: 0.369000
```



2.2 RMSProp and Adam

RMSProp [1] and Adam [2] are update rules that set per-parameter learning rates by using a running average of the second moments of gradients.

In the file `cs231n/optim.py`, implement the RMSProp update rule in the `rmsprop` function and implement the Adam update rule in the `adam` function, and check your implementations using the tests below.

NOTE: Please implement the *complete* Adam update rule (with the bias correction mechanism), not the first simplified version mentioned in the course notes.

[1] Tijmen Tieleman and Geoffrey Hinton. "Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude." COURSERA: Neural Networks for Machine Learning 4 (2012).

[2] Diederik Kingma and Jimmy Ba, "Adam: A Method for Stochastic Optimization", ICLR 2015.

[11]:
```
# Test RMSProp implementation
from cs231n.optim import rmsprop

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
cache = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'cache': cache}
next_w, _ = rmsprop(w, dw, config=config)

expected_next_w = np.asarray([
  [-0.39223849, -0.34037513, -0.28849239, -0.23659121, -0.18467247],
  [-0.132737,   -0.08078555, -0.02881884,  0.02316247,  0.07515774],
  [ 0.12716641,  0.17918792,  0.23122175,  0.28326742,  0.33532447],
  [ 0.38739248,  0.43947102,  0.49155973,  0.54365823,  0.59576619]])
expected_cache = np.asarray([
  [ 0.5976,      0.6126277,   0.6277108,   0.64284931,  0.65804321],
  [ 0.67329252,  0.68859723,  0.70395734,  0.71937285,  0.73484377],
  [ 0.75037008,  0.7659518,   0.78158892,  0.79728144,  0.81302936],
  [ 0.82883269,  0.84469141,  0.86060554,  0.87657507,  0.8926    ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('cache error: ', rel_error(expected_cache, config['cache']))
```

```
next_w error:  9.524687511038133e-08
cache error:  2.6477955807156126e-09
```

[12]:
```
# Test Adam implementation
from cs231n.optim import adam

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
m = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)
v = np.linspace(0.7, 0.5, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'm': m, 'v': v, 't': 5}
next_w, _ = adam(w, dw, config=config)

expected_next_w = np.asarray([
  [-0.40094747, -0.34836187, -0.29577703, -0.24319299, -0.19060977],
  [-0.1380274,  -0.08544591, -0.03286534,  0.01971428,  0.0722929],
  [ 0.1248705,   0.17744702,  0.23002243,  0.28259667,  0.33516969],
  [ 0.38774145,  0.44031188,  0.49288093,  0.54544852,  0.59801459]])
```

```
expected_v = np.asarray([
  [ 0.69966,     0.68908382,  0.67851319,  0.66794809,  0.65738853,],
  [ 0.64683452,  0.63628604,  0.6257431,   0.61520571,  0.60467385,],
  [ 0.59414753,  0.58362676,  0.57311152,  0.56260183,  0.55209767,],
  [ 0.54159906,  0.53110598,  0.52061845,  0.51013645,  0.49966,   ]])
expected_m = np.asarray([
  [ 0.48,        0.49947368,  0.51894737,  0.53842105,  0.55789474],
  [ 0.57736842,  0.59684211,  0.61631579,  0.63578947,  0.65526316],
  [ 0.67473684,  0.69421053,  0.71368421,  0.73315789,  0.75263158],
  [ 0.77210526,  0.79157895,  0.81105263,  0.83052632,  0.85      ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('v error: ', rel_error(expected_v, config['v']))
print('m error: ', rel_error(expected_m, config['m']))
```

```
next_w error:  1.1395691798535431e-07
v error:  4.208314038113071e-09
m error:  4.214963193114416e-09
```

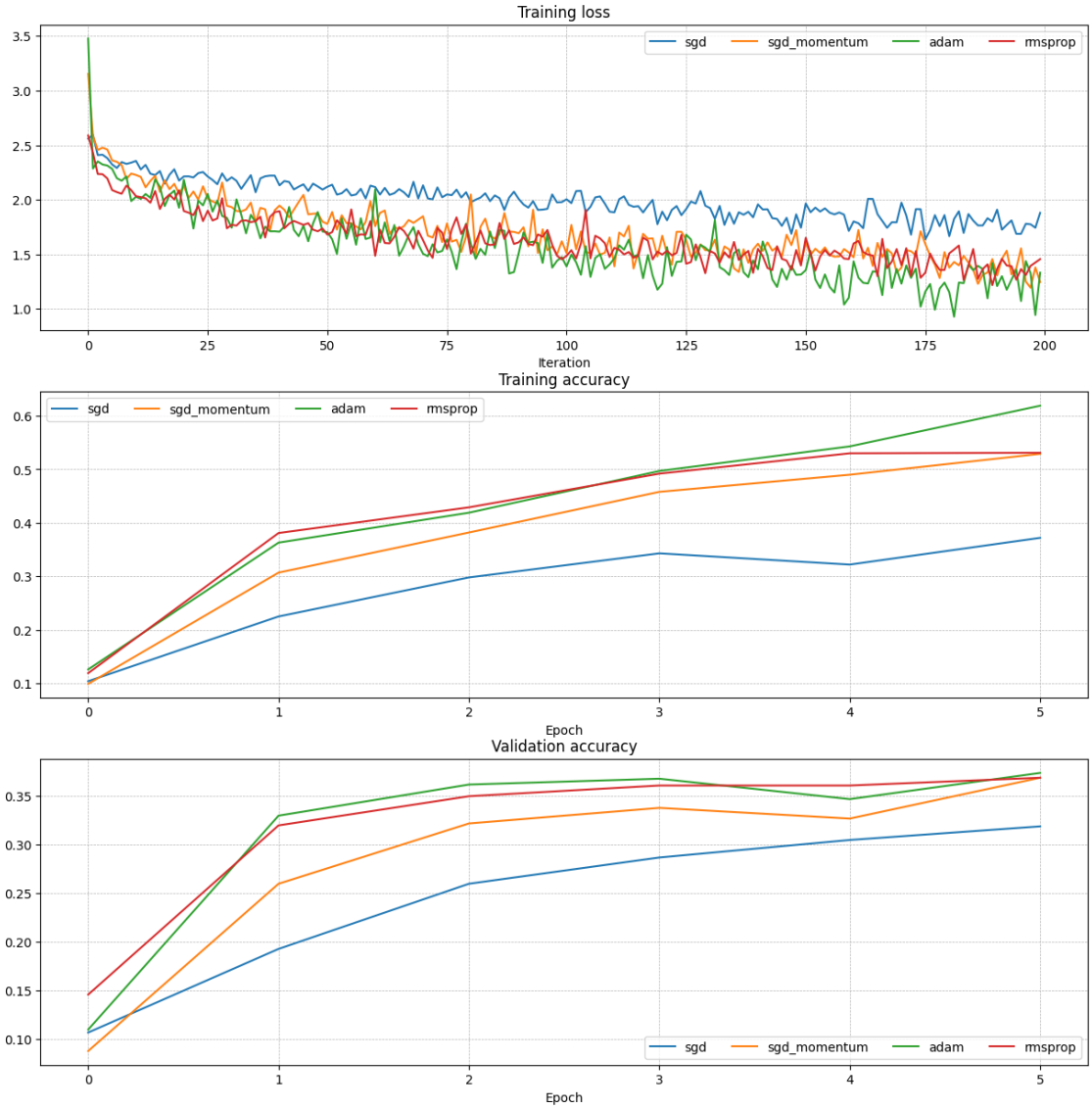
Once you have debugged your RMSProp and Adam implementations, run the following to train a pair of deep networks using these new update rules:

[13]:
```
learning_rates = {'rmsprop': 1e-4, 'adam': 1e-3}
for update_rule in ['adam', 'rmsprop']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )
    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': learning_rates[update_rule]},
        verbose=True
    )
    solvers[update_rule] = solver
    solver.train()
    print()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
```

```
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"{update_rule}")

for ax in axes:
    ax.legend(loc='best', ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()
```

```
Running with  adam
(Iteration 1 / 200) loss: 3.476928
(Epoch 0 / 5) train acc: 0.126000; val_acc: 0.110000
(Iteration 11 / 200) loss: 2.027712
(Iteration 21 / 200) loss: 2.183357
(Iteration 31 / 200) loss: 1.744257
(Epoch 1 / 5) train acc: 0.363000; val_acc: 0.330000
(Iteration 41 / 200) loss: 1.707951
(Iteration 51 / 200) loss: 1.703835
(Iteration 61 / 200) loss: 2.094758
(Iteration 71 / 200) loss: 1.505557
(Epoch 2 / 5) train acc: 0.419000; val_acc: 0.362000
(Iteration 81 / 200) loss: 1.594431
(Iteration 91 / 200) loss: 1.511452
(Iteration 101 / 200) loss: 1.389237
(Iteration 111 / 200) loss: 1.463575
(Epoch 3 / 5) train acc: 0.497000; val_acc: 0.368000
(Iteration 121 / 200) loss: 1.231313
(Iteration 131 / 200) loss: 1.520199
(Iteration 141 / 200) loss: 1.363221
(Iteration 151 / 200) loss: 1.355143
(Epoch 4 / 5) train acc: 0.543000; val_acc: 0.347000
(Iteration 161 / 200) loss: 1.436401
(Iteration 171 / 200) loss: 1.231426
(Iteration 181 / 200) loss: 1.153575
(Iteration 191 / 200) loss: 1.209479
(Epoch 5 / 5) train acc: 0.619000; val_acc: 0.374000

Running with  rmsprop
(Iteration 1 / 200) loss: 2.589166
(Epoch 0 / 5) train acc: 0.119000; val_acc: 0.146000
(Iteration 11 / 200) loss: 2.032921
```

```
(Iteration 21 / 200) loss: 1.897277
(Iteration 31 / 200) loss: 1.770793
(Epoch 1 / 5) train acc: 0.381000; val_acc: 0.320000
(Iteration 41 / 200) loss: 1.895731
(Iteration 51 / 200) loss: 1.681091
(Iteration 61 / 200) loss: 1.487204
(Iteration 71 / 200) loss: 1.629973
(Epoch 2 / 5) train acc: 0.429000; val_acc: 0.350000
(Iteration 81 / 200) loss: 1.506686
(Iteration 91 / 200) loss: 1.610742
(Iteration 101 / 200) loss: 1.486124
(Iteration 111 / 200) loss: 1.559454
(Epoch 3 / 5) train acc: 0.492000; val_acc: 0.361000
(Iteration 121 / 200) loss: 1.497406
(Iteration 131 / 200) loss: 1.530736
(Iteration 141 / 200) loss: 1.550957
(Iteration 151 / 200) loss: 1.652046
(Epoch 4 / 5) train acc: 0.530000; val_acc: 0.361000
(Iteration 161 / 200) loss: 1.599574
(Iteration 171 / 200) loss: 1.401073
(Iteration 181 / 200) loss: 1.509365
(Iteration 191 / 200) loss: 1.365773
(Epoch 5 / 5) train acc: 0.531000; val_acc: 0.369000
```



2.3 Inline Question 2:

AdaGrad, like Adam, is a per-parameter optimization method that uses the following update rule:

```
cache += dw**2
w += - learning_rate * dw / (np.sqrt(cache) + eps)
```

John notices that when he was training a network with AdaGrad that the updates became very small, and that his network was learning slowly. Using your knowledge of the AdaGrad update rule, why do you think the updates would become very small? Would Adam have the same issue?

2.4 Answer:

In AdaGrad, `cache` is a running **sum** of squared gradients: `cache += dw**2` has no decay or forgetting factor, so `cache` grows monotonically over the course of training. Since the effective step size is `learning_rate * dw / (sqrt(cache) + eps)`, the denominator keeps increasing, so every parameter's effective learning rate decays toward zero as training proceeds. Parameters that consistently receive large gradients accumulate squared-gradient mass the fastest, so their updates shrink first — that is why John's updates became vanishingly small and learning slowed down.

Adam does **not** have the same issue. Adam's cache is an exponentially weighted **moving average**, `v = beta2 * v + (1 - beta2) * dw**2`, rather than a plain sum: old squared gradients are exponentially forgotten, so `v` stays on the same scale as the *recent* gradients and never grows without bound. Furthermore, Adam divides by the bias-corrected first moment as well, so the update `m_hat / (sqrt(v_hat) + eps)` is essentially a bounded signed ratio (magnitude 1), which makes the per-step size roughly on the order of `learning_rate` no matter how many iterations have passed. In short: AdaGrad's non-decaying accumulator drives its effective learning rate monotonically to zero, while Adam's moving-average formulation keeps the step size stable throughout training.

3 Train a Good Model!

Train the best fully connected model that you can on CIFAR-10, storing your best model in the `best_model` variable. We require you to get at least 50% accuracy on the validation set using a fully connected network.

If you are careful it should be possible to get accuracies above 55%, but we don't require it for this part and won't assign extra credit for doing so. Later in the next assignment, we will ask you to train the best convolutional network that you can on CIFAR-10, and we would prefer that you spend your effort working on convolutional networks rather than fully connected networks.

Note: In the next assignment, you will learn techniques like BatchNormalization and Dropout which can help you train powerful models.

[14]:
```
best_model = None

################################################################################
# TODO: Train the best FullyConnectedNet that you can on CIFAR-10. You might   #
# find batch/layer normalization and dropout useful. Store your best model in  #
# the best_model variable.                                                     #
################################################################################
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****

best_val = -1
best_config = {}

#   4000
for lr in [1e-3, 5e-4]:
    for reg in [1e-4, 1e-3]:
        for kr in [0.9, 1.0]:
            model = FullyConnectedNet(
```

```
                [500, 100], weight_scale=1e-2, reg=reg,
                dropout_keep_ratio=kr, normalization="batchnorm", dtype=np.
 ↪float32)
            solver = Solver(model, small_data, num_epochs=5, batch_size=100,
                            update_rule="adam",
                            optim_config={"learning_rate": lr}, verbose=False)
            solver.train()
            print(f"lr={lr} reg={reg} keep={kr} -> val_acc={solver.best_val_acc:
 ↪.4f}")
            if solver.best_val_acc > best_val:
                best_val = solver.best_val_acc
                best_config = {"lr": lr, "reg": reg, "kr": kr}

#
best_model = FullyConnectedNet(
    [500, 100], weight_scale=1e-2, reg=best_config["reg"],
    dropout_keep_ratio=best_config["kr"], normalization="batchnorm", dtype=np.
 ↪float32)
solver = Solver(best_model, data, num_epochs=15, batch_size=256,
                update_rule="adam",
                optim_config={"learning_rate": best_config["lr"]}, verbose=True)
solver.train()


# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
################################################################################
#                              END OF YOUR CODE                                #
################################################################################
```

```
lr=0.001 reg=0.0001 keep=0.9 -> val_acc=0.4220
lr=0.001 reg=0.0001 keep=1.0 -> val_acc=0.4130
lr=0.001 reg=0.001 keep=0.9 -> val_acc=0.3890
lr=0.001 reg=0.001 keep=1.0 -> val_acc=0.4010
lr=0.0005 reg=0.0001 keep=0.9 -> val_acc=0.3990
lr=0.0005 reg=0.0001 keep=1.0 -> val_acc=0.4260
lr=0.0005 reg=0.001 keep=0.9 -> val_acc=0.4260
lr=0.0005 reg=0.001 keep=1.0 -> val_acc=0.4280
(Iteration 1 / 2865) loss: 2.388986
(Epoch 0 / 15) train acc: 0.248000; val_acc: 0.231000
(Iteration 11 / 2865) loss: 2.195182
(Iteration 21 / 2865) loss: 2.143569
(Iteration 31 / 2865) loss: 2.036120
(Iteration 41 / 2865) loss: 1.907103
(Iteration 51 / 2865) loss: 1.885541
(Iteration 61 / 2865) loss: 1.895978
(Iteration 71 / 2865) loss: 1.786040
(Iteration 81 / 2865) loss: 1.807802
```

```
(Iteration 91 / 2865) loss: 1.705372
(Iteration 101 / 2865) loss: 1.755347
(Iteration 111 / 2865) loss: 1.650800
(Iteration 121 / 2865) loss: 1.634307
(Iteration 131 / 2865) loss: 1.690166
(Iteration 141 / 2865) loss: 1.629953
(Iteration 151 / 2865) loss: 1.598872
(Iteration 161 / 2865) loss: 1.573275
(Iteration 171 / 2865) loss: 1.566933
(Iteration 181 / 2865) loss: 1.668535
(Iteration 191 / 2865) loss: 1.696813
(Epoch 1 / 15) train acc: 0.521000; val_acc: 0.478000
(Iteration 201 / 2865) loss: 1.559045
(Iteration 211 / 2865) loss: 1.554240
(Iteration 221 / 2865) loss: 1.644890
(Iteration 231 / 2865) loss: 1.501249
(Iteration 241 / 2865) loss: 1.507025
(Iteration 251 / 2865) loss: 1.530621
(Iteration 261 / 2865) loss: 1.530867
(Iteration 271 / 2865) loss: 1.486382
(Iteration 281 / 2865) loss: 1.454265
(Iteration 291 / 2865) loss: 1.385851
(Iteration 301 / 2865) loss: 1.385188
(Iteration 311 / 2865) loss: 1.441938
(Iteration 321 / 2865) loss: 1.378103
(Iteration 331 / 2865) loss: 1.425606
(Iteration 341 / 2865) loss: 1.300716
(Iteration 351 / 2865) loss: 1.454054
(Iteration 361 / 2865) loss: 1.311827
(Iteration 371 / 2865) loss: 1.364827
(Iteration 381 / 2865) loss: 1.425396
(Epoch 2 / 15) train acc: 0.544000; val_acc: 0.499000
(Iteration 391 / 2865) loss: 1.329322
(Iteration 401 / 2865) loss: 1.333690
(Iteration 411 / 2865) loss: 1.360658
(Iteration 421 / 2865) loss: 1.499543
(Iteration 431 / 2865) loss: 1.510991
(Iteration 441 / 2865) loss: 1.372442
(Iteration 451 / 2865) loss: 1.432465
(Iteration 461 / 2865) loss: 1.390876
(Iteration 471 / 2865) loss: 1.367633
(Iteration 481 / 2865) loss: 1.453895
(Iteration 491 / 2865) loss: 1.289503
(Iteration 501 / 2865) loss: 1.332748
(Iteration 511 / 2865) loss: 1.340607
(Iteration 521 / 2865) loss: 1.274221
(Iteration 531 / 2865) loss: 1.294120
(Iteration 541 / 2865) loss: 1.367879
```

```
(Iteration 551 / 2865) loss: 1.320315
(Iteration 561 / 2865) loss: 1.281360
(Iteration 571 / 2865) loss: 1.443321
(Epoch 3 / 15) train acc: 0.584000; val_acc: 0.531000
(Iteration 581 / 2865) loss: 1.339678
(Iteration 591 / 2865) loss: 1.331857
(Iteration 601 / 2865) loss: 1.339622
(Iteration 611 / 2865) loss: 1.402860
(Iteration 621 / 2865) loss: 1.391565
(Iteration 631 / 2865) loss: 1.434753
(Iteration 641 / 2865) loss: 1.413692
(Iteration 651 / 2865) loss: 1.306547
(Iteration 661 / 2865) loss: 1.244099
(Iteration 671 / 2865) loss: 1.256189
(Iteration 681 / 2865) loss: 1.388564
(Iteration 691 / 2865) loss: 1.291507
(Iteration 701 / 2865) loss: 1.373560
(Iteration 711 / 2865) loss: 1.210897
(Iteration 721 / 2865) loss: 1.329872
(Iteration 731 / 2865) loss: 1.394847
(Iteration 741 / 2865) loss: 1.304091
(Iteration 751 / 2865) loss: 1.174226
(Iteration 761 / 2865) loss: 1.321914
(Epoch 4 / 15) train acc: 0.607000; val_acc: 0.521000
(Iteration 771 / 2865) loss: 1.229456
(Iteration 781 / 2865) loss: 1.208158
(Iteration 791 / 2865) loss: 1.295637
(Iteration 801 / 2865) loss: 1.312803
(Iteration 811 / 2865) loss: 1.300636
(Iteration 821 / 2865) loss: 1.296999
(Iteration 831 / 2865) loss: 1.208425
(Iteration 841 / 2865) loss: 1.239316
(Iteration 851 / 2865) loss: 1.140845
(Iteration 861 / 2865) loss: 1.157783
(Iteration 871 / 2865) loss: 1.254395
(Iteration 881 / 2865) loss: 1.185077
(Iteration 891 / 2865) loss: 1.257198
(Iteration 901 / 2865) loss: 1.164916
(Iteration 911 / 2865) loss: 1.211355
(Iteration 921 / 2865) loss: 1.237634
(Iteration 931 / 2865) loss: 1.209776
(Iteration 941 / 2865) loss: 1.145301
(Iteration 951 / 2865) loss: 1.203613
(Epoch 5 / 15) train acc: 0.592000; val_acc: 0.532000
(Iteration 961 / 2865) loss: 1.348463
(Iteration 971 / 2865) loss: 1.263436
(Iteration 981 / 2865) loss: 1.235278
(Iteration 991 / 2865) loss: 1.247433
```

```
(Iteration 1001 / 2865) loss: 1.304801
(Iteration 1011 / 2865) loss: 1.220770
(Iteration 1021 / 2865) loss: 1.199839
(Iteration 1031 / 2865) loss: 1.213709
(Iteration 1041 / 2865) loss: 1.170236
(Iteration 1051 / 2865) loss: 1.332385
(Iteration 1061 / 2865) loss: 1.159760
(Iteration 1071 / 2865) loss: 1.193129
(Iteration 1081 / 2865) loss: 1.195890
(Iteration 1091 / 2865) loss: 1.245018
(Iteration 1101 / 2865) loss: 1.174564
(Iteration 1111 / 2865) loss: 1.112661
(Iteration 1121 / 2865) loss: 1.179812
(Iteration 1131 / 2865) loss: 1.232088
(Iteration 1141 / 2865) loss: 1.122386
(Epoch 6 / 15) train acc: 0.617000; val_acc: 0.549000
(Iteration 1151 / 2865) loss: 1.213578
(Iteration 1161 / 2865) loss: 1.216424
(Iteration 1171 / 2865) loss: 1.273848
(Iteration 1181 / 2865) loss: 1.220045
(Iteration 1191 / 2865) loss: 1.223900
(Iteration 1201 / 2865) loss: 1.265529
(Iteration 1211 / 2865) loss: 1.212001
(Iteration 1221 / 2865) loss: 1.299984
(Iteration 1231 / 2865) loss: 1.197779
(Iteration 1241 / 2865) loss: 1.123634
(Iteration 1251 / 2865) loss: 1.247053
(Iteration 1261 / 2865) loss: 1.292774
(Iteration 1271 / 2865) loss: 1.218552
(Iteration 1281 / 2865) loss: 1.130407
(Iteration 1291 / 2865) loss: 1.274040
(Iteration 1301 / 2865) loss: 1.208832
(Iteration 1311 / 2865) loss: 1.199156
(Iteration 1321 / 2865) loss: 1.086179
(Iteration 1331 / 2865) loss: 1.179997
(Epoch 7 / 15) train acc: 0.641000; val_acc: 0.540000
(Iteration 1341 / 2865) loss: 1.174941
(Iteration 1351 / 2865) loss: 1.129997
(Iteration 1361 / 2865) loss: 1.228351
(Iteration 1371 / 2865) loss: 1.123667
(Iteration 1381 / 2865) loss: 1.116775
(Iteration 1391 / 2865) loss: 1.180323
(Iteration 1401 / 2865) loss: 1.249428
(Iteration 1411 / 2865) loss: 1.098521
(Iteration 1421 / 2865) loss: 1.213519
(Iteration 1431 / 2865) loss: 1.175576
(Iteration 1441 / 2865) loss: 1.070313
(Iteration 1451 / 2865) loss: 1.213312
```

```
(Iteration 1461 / 2865) loss: 1.064497
(Iteration 1471 / 2865) loss: 1.074620
(Iteration 1481 / 2865) loss: 1.097714
(Iteration 1491 / 2865) loss: 1.169443
(Iteration 1501 / 2865) loss: 1.163499
(Iteration 1511 / 2865) loss: 1.221545
(Iteration 1521 / 2865) loss: 1.095853
(Epoch 8 / 15) train acc: 0.670000; val_acc: 0.563000
(Iteration 1531 / 2865) loss: 1.188904
(Iteration 1541 / 2865) loss: 1.181271
(Iteration 1551 / 2865) loss: 1.155711
(Iteration 1561 / 2865) loss: 1.145053
(Iteration 1571 / 2865) loss: 1.114176
(Iteration 1581 / 2865) loss: 1.156528
(Iteration 1591 / 2865) loss: 1.226251
(Iteration 1601 / 2865) loss: 1.215919
(Iteration 1611 / 2865) loss: 1.082898
(Iteration 1621 / 2865) loss: 1.182094
(Iteration 1631 / 2865) loss: 1.212647
(Iteration 1641 / 2865) loss: 1.099059
(Iteration 1651 / 2865) loss: 1.102074
(Iteration 1661 / 2865) loss: 1.085756
(Iteration 1671 / 2865) loss: 1.182973
(Iteration 1681 / 2865) loss: 1.098096
(Iteration 1691 / 2865) loss: 1.118520
(Iteration 1701 / 2865) loss: 1.155318
(Iteration 1711 / 2865) loss: 1.090740
(Epoch 9 / 15) train acc: 0.671000; val_acc: 0.555000
(Iteration 1721 / 2865) loss: 1.067544
(Iteration 1731 / 2865) loss: 1.161486
(Iteration 1741 / 2865) loss: 1.107172
(Iteration 1751 / 2865) loss: 1.153270
(Iteration 1761 / 2865) loss: 1.163558
(Iteration 1771 / 2865) loss: 1.050658
(Iteration 1781 / 2865) loss: 1.146121
(Iteration 1791 / 2865) loss: 1.120377
(Iteration 1801 / 2865) loss: 1.123428
(Iteration 1811 / 2865) loss: 1.153973
(Iteration 1821 / 2865) loss: 1.118015
(Iteration 1831 / 2865) loss: 1.213810
(Iteration 1841 / 2865) loss: 1.195832
(Iteration 1851 / 2865) loss: 1.000480
(Iteration 1861 / 2865) loss: 0.975224
(Iteration 1871 / 2865) loss: 1.100772
(Iteration 1881 / 2865) loss: 1.161238
(Iteration 1891 / 2865) loss: 1.083992
(Iteration 1901 / 2865) loss: 1.071284
(Epoch 10 / 15) train acc: 0.684000; val_acc: 0.543000
```

```
(Iteration 1911 / 2865) loss: 1.112223
(Iteration 1921 / 2865) loss: 1.076676
(Iteration 1931 / 2865) loss: 1.150425
(Iteration 1941 / 2865) loss: 1.023267
(Iteration 1951 / 2865) loss: 1.023026
(Iteration 1961 / 2865) loss: 1.017423
(Iteration 1971 / 2865) loss: 1.141696
(Iteration 1981 / 2865) loss: 1.064955
(Iteration 1991 / 2865) loss: 1.070845
(Iteration 2001 / 2865) loss: 1.062153
(Iteration 2011 / 2865) loss: 1.048662
(Iteration 2021 / 2865) loss: 1.053955
(Iteration 2031 / 2865) loss: 1.062546
(Iteration 2041 / 2865) loss: 1.088792
(Iteration 2051 / 2865) loss: 1.124781
(Iteration 2061 / 2865) loss: 1.074188
(Iteration 2071 / 2865) loss: 1.033580
(Iteration 2081 / 2865) loss: 1.061078
(Iteration 2091 / 2865) loss: 0.967455
(Iteration 2101 / 2865) loss: 1.006272
(Epoch 11 / 15) train acc: 0.676000; val_acc: 0.561000
(Iteration 2111 / 2865) loss: 1.099474
(Iteration 2121 / 2865) loss: 1.083827
(Iteration 2131 / 2865) loss: 1.115209
(Iteration 2141 / 2865) loss: 1.014202
(Iteration 2151 / 2865) loss: 1.030763
(Iteration 2161 / 2865) loss: 0.996820
(Iteration 2171 / 2865) loss: 1.037732
(Iteration 2181 / 2865) loss: 1.067713
(Iteration 2191 / 2865) loss: 0.992415
(Iteration 2201 / 2865) loss: 1.022544
(Iteration 2211 / 2865) loss: 1.112435
(Iteration 2221 / 2865) loss: 0.995825
(Iteration 2231 / 2865) loss: 1.121277
(Iteration 2241 / 2865) loss: 1.061673
(Iteration 2251 / 2865) loss: 1.225536
(Iteration 2261 / 2865) loss: 0.908792
(Iteration 2271 / 2865) loss: 0.978359
(Iteration 2281 / 2865) loss: 0.991000
(Iteration 2291 / 2865) loss: 1.035101
(Epoch 12 / 15) train acc: 0.700000; val_acc: 0.560000
(Iteration 2301 / 2865) loss: 0.996407
(Iteration 2311 / 2865) loss: 0.965859
(Iteration 2321 / 2865) loss: 0.965944
(Iteration 2331 / 2865) loss: 0.988567
(Iteration 2341 / 2865) loss: 1.025768
(Iteration 2351 / 2865) loss: 1.011742
(Iteration 2361 / 2865) loss: 0.962072
```

```
(Iteration 2371 / 2865) loss: 1.026439
(Iteration 2381 / 2865) loss: 1.067225
(Iteration 2391 / 2865) loss: 1.096334
(Iteration 2401 / 2865) loss: 1.089098
(Iteration 2411 / 2865) loss: 1.003305
(Iteration 2421 / 2865) loss: 1.206238
(Iteration 2431 / 2865) loss: 1.092600
(Iteration 2441 / 2865) loss: 0.965314
(Iteration 2451 / 2865) loss: 1.093791
(Iteration 2461 / 2865) loss: 1.001604
(Iteration 2471 / 2865) loss: 0.961262
(Iteration 2481 / 2865) loss: 0.973718
(Epoch 13 / 15) train acc: 0.693000; val_acc: 0.546000
(Iteration 2491 / 2865) loss: 0.980435
(Iteration 2501 / 2865) loss: 0.967596
(Iteration 2511 / 2865) loss: 0.995178
(Iteration 2521 / 2865) loss: 0.993491
(Iteration 2531 / 2865) loss: 1.063977
(Iteration 2541 / 2865) loss: 1.080938
(Iteration 2551 / 2865) loss: 0.955614
(Iteration 2561 / 2865) loss: 1.038391
(Iteration 2571 / 2865) loss: 1.081631
(Iteration 2581 / 2865) loss: 1.047810
(Iteration 2591 / 2865) loss: 1.040259
(Iteration 2601 / 2865) loss: 1.039519
(Iteration 2611 / 2865) loss: 1.096414
(Iteration 2621 / 2865) loss: 0.939155
(Iteration 2631 / 2865) loss: 1.027030
(Iteration 2641 / 2865) loss: 1.003374
(Iteration 2651 / 2865) loss: 1.057313
(Iteration 2661 / 2865) loss: 1.040023
(Iteration 2671 / 2865) loss: 0.955017
(Epoch 14 / 15) train acc: 0.700000; val_acc: 0.563000
(Iteration 2681 / 2865) loss: 1.024183
(Iteration 2691 / 2865) loss: 1.095874
(Iteration 2701 / 2865) loss: 1.050289
(Iteration 2711 / 2865) loss: 0.838899
(Iteration 2721 / 2865) loss: 0.974209
(Iteration 2731 / 2865) loss: 0.921370
(Iteration 2741 / 2865) loss: 1.019835
(Iteration 2751 / 2865) loss: 0.957431
(Iteration 2761 / 2865) loss: 0.998109
(Iteration 2771 / 2865) loss: 0.986145
(Iteration 2781 / 2865) loss: 0.996377
(Iteration 2791 / 2865) loss: 0.979749
(Iteration 2801 / 2865) loss: 0.898654
(Iteration 2811 / 2865) loss: 0.951078
(Iteration 2821 / 2865) loss: 1.053541
```

```
(Iteration 2831 / 2865) loss: 1.035793
(Iteration 2841 / 2865) loss: 1.110621
(Iteration 2851 / 2865) loss: 0.949097
(Iteration 2861 / 2865) loss: 1.016207
(Epoch 15 / 15) train acc: 0.722000; val_acc: 0.565000
```

4 Test Your Model!

Run your best model on the validation and test sets. You should achieve at least 50% accuracy on the validation set and the test set.

[15]:
```
y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

```
Validation set accuracy:  0.565
Test set accuracy:  0.568
```